

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN - ĐIỆN TỬ

*



BÁO CÁO THỰC TẬP NGOÀI TRƯỜNG
FPGA Implementation of LDPC Decoder
Architecture for Wireless Communication Standards

GVHD : ThS. Nguyễn Trung Hiếu
HỌ VÀ TÊN Vũ Nhật Huy
MSSV : 2311267

Ho Chi Minh City, Ngày 16 tháng 8 năm 2026

Lời nói đầu

Em xin gửi lời cảm ơn chân thành và sự tri ân sâu sắc đối với thầy Nguyễn Trung Hiếu, giảng viên Bộ môn Điện tử trường Đại học Bách khoa - Đại học Quốc gia Thành phố Hồ Chí Minh, đã tạo điều kiện cho em có nhiều thời gian cho Báo cáo thực tập ngoài trường này. Và đồng thời em cũng xin chân thành cảm ơn thầy đã nhiệt tình hướng dẫn hướng dẫn giúp em hoàn thành tốt báo cáo này.

Trong quá trình học tập, cũng như là trong quá trình làm bài báo cáo, do điều kiện khó khăn khó tránh khỏi sai sót, rất mong Thầy có thể thông cảm. Đồng thời do trình độ lý luận cũng như kinh nghiệm thực tiễn còn hạn chế nên bài báo cáo không thể tránh khỏi những thiếu sót, em rất mong nhận được ý kiến đóng góp từ Thầy để em học thêm được nhiều kinh nghiệm và sẽ hoàn thành tốt những Đồ án, Luận văn tốt nghiệp trong tương lai.

EM xin chân thành cảm ơn Thầy!

Chúc Thầy nhiều sức khỏe.

Mục lục

1	Giới thiệu	1
1.1	Động lực nghiên cứu	1
1.2	Tình hình nghiên cứu	1
1.3	Cấu trúc báo cáo	2
2	Cơ sở lý thuyết	3
2.1	Tổng quan về Mã Kiểm Tra Chặn Lẻ Mật Độ Thấp (LDPC)	3
2.2	Thuật Toán Giải Mã Min-Sum	3
2.2.1	Biểu thức rút gọn phục vụ tối ưu hóa phần cứng	5
2.2.2	Ví dụ tổng quát về thuật toán Min-Sum	5
3	Thiết kế và thực hiện phần cứng	7
3.1	Tổng quan hoạt động hệ thống	7
3.2	Khối BRAM	8
3.2.1	Khối BRAM APP	8
3.2.2	Khối BRAM CTV	9
3.3	Khối Iterations Counter	10
3.4	Khối Operating Mode Selection	11
3.5	Khối VTC Calculation	12
3.6	Khối Min/Submin Calculation	14
3.7	Khối CTV & APP Calculation	18
4	Thực hiện phần mềm và mô phỏng	19
4.1	Khối BRAM APP	19
4.2	Khối BRAM CTV	21
4.3	Khối Iteration Counter	24
4.4	Khối Mode Select	27
4.5	Khối VTC Calculation	30
4.6	Khối Min/Submin Calculation	33
4.7	Khối CTV APP Calculation	35
5	Kết luận và hướng phát triển	38
5.1	Kết luận	38
5.2	Định hướng phát triển	38

Danh sách hình vẽ

3.1	Kiến trúc bộ giải mã LDPC	7
3.2	Kiến trúc khối BRAM APP	8
3.3	Kiến trúc khối BRAM CTV	9
3.4	Kiến trúc khối Iterations Counter	10
3.5	Kiến trúc khối Iterations Counter	11
3.6	Kiến trúc khối Iterations Compare	12
3.7	Kiến trúc khối Mode Select	12
3.8	Kiến trúc khối VTC Calculation	13
3.9	Kiến trúc Bộ so sánh độ lớn số 6 bit	13
3.10	Kiến trúc phần tử con full adder 8 bit	14
3.11	Kiến trúc Khối ABS	15
3.12	Kiến trúc Khối SignsXoring	15
3.13	Kiến trúc phần tử con Comp min	16
3.14	Kiến trúc Khối Compare tree	16
3.15	Kiến trúc Khối Sign Insertion	17
3.16	Kiến trúc Khối CTV APP CALC	18
4.1	Dạng sóng của khối BRAM APP	19
4.2	Dạng sóng của khối BRAM CTV	21
4.3	Dạng sóng của khối Iteration Counter	24
4.4	Dạng sóng của khối Mode Select	27
4.5	Dạng sóng của khối VTC Calculation	30
4.6	Dạng sóng của khối Min Submin Calculation	33
4.7	Dạng sóng của khối CTV APP Calculation	35

Danh sách bảng

4.1	Tổng hợp thông số Diện tích và Công suất của khối BRAM APP qua các process corners.	20
4.2	Thống kê độ trễ các góc phân tích (PVT corners) cho khối bram_app. . .	20
4.3	Tổng hợp thông số Diện tích và Công suất của khối BRAM CTV qua các process corners.	22
4.4	Thống kê độ trễ các góc phân tích (PVT corners) cho khối bram_ctv. . . .	23
4.5	Tổng hợp thông số Diện tích và Công suất của khối Iteration Counter qua các process corners.	25
4.6	Thống kê độ trễ các góc phân tích (PVT corners) cho khối iteration_counter. .	26
4.7	Tổng hợp thông số Diện tích và Công suất của khối Mode Select qua các process corners.	28
4.8	Thống kê độ trễ các góc phân tích (PVT corners) cho khối mode_select. . .	29
4.9	Tổng hợp thông số Diện tích và Công suất của khối VTC Calculation qua các process corners.	31
4.10	Thống kê độ trễ các góc phân tích (PVT corners) cho khối vtc_calc. . . .	32
4.11	Tổng hợp thông số Diện tích và Công suất của khối Min/Submin Calculation qua các process corners.	34
4.12	Bảng tổng hợp thông số timing các góc phân tích (PVT corners) cho khối min_submin.	35
4.13	Tổng hợp thông số Diện tích và Công suất của khối Min/Submin Calculation qua các process corners.	36
4.14	Thống kê độ trễ các góc phân tích (PVT corners) cho khối ctv_app_calc. .	37

1. Giới thiệu

1.1 Động lực nghiên cứu

Mã kiểm tra chẵn lẻ mật độ thấp (Low-Density Parity-Check - LDPC) đóng vai trò quan trọng trong các hệ thống thông tin hiện đại nhờ khả năng sửa lỗi tối ưu. Cụ thể, các nghiên cứu [1] đã chỉ rằng các mã LDPC bất thường nếu được thiết kế tối ưu có thể đạt ngưỡng hiệu suất chỉ cách giới hạn Shannon một khoảng cực kỳ nhỏ là 0.0045 dB ở tỷ lệ lỗi bit $BER = 10^{-6}$. Thậm chí, trong các mô phỏng thực tế với chiều dài khối lên tới 10^7 thì các bộ giải mã này vẫn duy trì khả năng hoạt động cực kỳ ổn định và đạt mức cách giới hạn Shannon chỉ 0.04 dB ở mức $BER = 10^{-6}$. Nhờ đặc tính vượt trội này mà mã LDPC đã trở thành một phần không thể thiếu trong các chuẩn truyền thông không dây băng thông rộng phổ biến như IEEE 802.11n/ad/ay/ax (Wifi), IEEE 802.16e (WiMAX) và mạng di động thế hệ mới ETSI 5G.

Trong thực tế triển khai, các ứng dụng truyền thông hiện đại đòi hỏi tốc độ dữ liệu ngày càng cao và khả năng tương thích linh hoạt với nhiều tiêu chuẩn khác nhau. Tuy nhiên, phần lớn các kiến trúc giải mã song song hiện nay chỉ được thiết kế cố định cho một tiêu chuẩn truyền thông duy nhất hoặc một chiều dài từ mã cụ thể, dẫn đến sự thiếu linh hoạt. Bên cạnh đó, các nghiên cứu trước đây [2], [3], [4] thường lạm dụng tài nguyên bộ nhớ khối (BRAM) để lưu trữ các giá trị nội bộ mà chưa chú trọng đến việc tối ưu hóa cách thức lưu trữ ma trận kiểm tra chẵn lẻ thưa (Sparse Parity-Check Matrix), gây lãng phí tài nguyên phần cứng trên chip. Để giải quyết triệt để, kiến trúc bộ giải mã đề xuất chỉ lưu trữ ma trận kiểm tra chẵn lẻ cơ sở (base parity-check matrix) H_b . Khối dữ liệu này được nén thông qua 3 vector nội bộ: *vals* (chứa các phần tử có giá trị khác -1), *pos* (chứa chỉ số cột tương ứng) và *elementSize* (chứa số lượng phần tử khác -1 trong mỗi hàng). Phương pháp này giúp giảm thiểu đột phá dung lượng BRAM cần thiết.

Việc giải quyết bài toán tối ưu hóa tài nguyên lưu trữ đồng thời tăng cường khả năng cấu hình cho bộ giải mã là vô cùng cấp thiết. Một kiến trúc giải mã QC-LDPC có thể tái cấu hình linh hoạt sẽ giúp giảm thiểu chi phí thiết kế phần cứng và đẩy nhanh tốc độ xử lý. Kết quả của nghiên cứu này có thể ứng dụng trực tiếp vào các hệ thống hạ tầng viễn thông không dây tốc độ cao và các thiết bị lưu trữ thế hệ mới, đáp ứng đồng thời cả hai tiêu chuẩn phổ biến là WiMAX và WiFi trên cùng một nền tảng phần cứng.

1.2 Tình hình nghiên cứu

Trong kỷ nguyên truyền thông tốc độ cao, yêu cầu khắt khe về băng thông đã thúc đẩy vô số công trình nghiên cứu về kiến trúc giải mã LDPC song song được công bố trong thập kỷ qua. Dù các kiến trúc này đã chứng minh được hiệu năng xử lý ở các mức độ nhất định, chúng vẫn tồn tại những nhược điểm về kiến trúc lưu trữ và độ linh hoạt, cụ thể:

- Hạn chế về tính linh hoạt phần cứng. Tiêu biểu như nghiên cứu của C. Kun [2], kiến trúc giải mã được thiết kế cố định cho một khung tiêu chuẩn cụ thể. Điều này đồng nghĩa với việc khi hệ thống cần chuyển đổi từ giao thức WiFi sang WiMAX thì toàn bộ phần cứng phải được thiết kế và tổng hợp lại từ đầu, gây lãng phí thời gian và tốn kém chi phí triển khai.
- Ngoài ra trước đây có khá nhiều thiết kế đã áp dụng phương pháp lưu trữ dữ liệu tính toán nội bộ trực tiếp lên BRAM mà không nén dữ liệu. Nghiên cứu của S.

Mhaske [3] là một ví dụ minh họa rõ nét. Dù kiến trúc giải mã thuật toán này đạt được tốc độ bằng thông lên tới $2.48Gb/s$, nhưng lại tiêu tốn một lượng BRAM do không tối ưu hóa việc lưu trữ ma trận thưa. Việc lạm dụng BRAM theo cách truyền thống không chỉ gây lãng phí mà còn làm phức tạp hóa quá trình truy xuất dữ liệu song song, dễ tạo ra các điểm nghẽn bằng thông khi mở rộng số lượng lõi giải mã.

- Mặc dù đã có một số ít các công trình nghiên cứu bắt đầu chú ý đến việc tối ưu hóa bộ nhớ, như công trình của S. Nimara [4] đề xuất một kiến trúc giải mã đa từ mã với hiệu suất sử dụng BRAM tốt hơn. Tuy nhiên thì công trình này vẫn thiếu đi khả năng tham số hóa để hỗ trợ đa chuẩn truyền thông bằng thông rộng trên cùng một thiết kế nguyên khối. Ngoại trừ số ít nghiên cứu cung cấp được bộ giải mã linh hoạt nhưng phần lớn các kiến trúc hiện hành vẫn chưa giải quyết được bài toán cân bằng giữa tối ưu bộ nhớ và tính linh hoạt đa chuẩn.

Chính từ những nhược điểm này thì việc phát triển một kiến trúc giải mã QC-LDPC kết hợp sử dụng thuật toán Min-Sum và có khả năng cấu hình bằng tham số là vô cùng cấp thiết. Kiến trúc đề xuất trong báo cáo này kỳ vọng giải quyết toàn diện các nhược điểm trên, đạt được bằng thông giải mã lên tới $1.2Gbps$ ở tần số hoạt động $240MHz$ mà vẫn giữ mức tiêu thụ tài nguyên phần cứng cực kỳ tối ưu trên FPGA.

1.3 Cấu trúc báo cáo

Tổng quan toàn bộ quá trình thực hiện và triển khai bộ giải mã LDPC của bản báo cáo này được cấu trúc thành 7 chương như sau:

- Chương 1: Giới thiệu.
- Chương 2: Cơ sở lý thuyết.
- Chương 3: Thiết kế và thực hiện phần mềm.
- Chương 4: Thiết kế và thực hiện phần cứng.
- Chương 5: Mô phỏng phần cứng.
- Chương 6: Kết quả thực hiện.
- Chương 7: Kết luận và hướng phát triển.

2. Cơ sở lý thuyết

2.1 Tổng quan về Mã Kiểm Tra Chẵn Lẻ Mật Độ Thấp (LDPC)

LDPC là một lớp mã khối tuyến tính được phát triển ban đầu bởi Gallager vào năm 1963 [5]. Đặc tính tuyến tính này cho phép mã được áp dụng trực tiếp lên các khối dữ liệu nguồn để thực hiện chức năng phát hiện và sửa lỗi trên kênh truyền nhiễu. Dù từng bị lãng quên trong nhiều thập kỷ do giới hạn phần cứng, mã LDPC đã trở lại mạnh mẽ sau khi được MacKay và Neal khám phá lại vào cuối những năm 1990 [6].

Khái niệm mật độ thấp xuất phát từ đặc điểm cấu trúc hình học của ma trận kiểm tra chẵn lẻ H (parity-check matrix) đại diện cho mã. Ma trận H là một ma trận thưa, nghĩa là số lượng phần tử có giá trị bằng '1' chiếm tỷ lệ rất nhỏ so với số lượng phần tử mang giá trị bằng '0' bên trong ma trận.

Một mã khối tuyến tính LDPC đều loại (N, K) được biểu diễn cấu trúc thông qua ma trận kiểm tra H có kích thước $M \times N$, với $M \geq N - K$ hàng và N cột. Khi biểu diễn dưới dạng đồ thị hai phía Tanner, cấu trúc này bao gồm 2 thành phần chính sau:

- Các nút kiểm tra (Check Nodes - CN): Đại diện bởi M hàng của ma trận H , đóng vai trò kiểm tra tính ràng buộc chẵn lẻ của các phương trình toán học.
- Các nút biến (Variable Nodes - VN): Đại diện bởi N cột của ma trận H , tương ứng trực tiếp với các phần tử bit cấu thành nên một từ mã.

Trong các ứng dụng thực tế, các tiêu chuẩn truyền thông không dây băng thông rộng như WiFi và WiMAX không sử dụng mã LDPC mà sử dụng một biến thể gọi là mã LDPC tuần hoàn (QC-LDPC). Đây là nền tảng toán học cốt lõi cho phép triển khai kiến trúc giải mã song song trên phần cứng FPGA. Đặc tính tựa vòng thể hiện ở chỗ ma trận kiểm tra chẵn lẻ H được cấu trúc từ nhiều ma trận con vuông. Các ma trận con này được tạo ra bằng cách dịch vòng một ma trận đơn vị theo các giá trị dịch chuyển xác định.

Nhờ tính chất này, quá trình triển khai phần cứng không cần phải lưu trữ toàn bộ ma trận H khổng lồ. Thay vào đó, bộ giải mã chỉ cần lưu trữ ma trận kiểm tra chẵn lẻ cơ sở H_b (base parity-check matrix). Việc biểu diễn H_b giúp giảm thiểu đáng kể tài nguyên bộ nhớ BRAM cần thiết trên chip. Bên cạnh đó, kích thước của các ma trận con tựa vòng được ký hiệu là hệ số khai triển từ mã z (codeword circulant size), quyết định trực tiếp đến mức độ song song hóa của bộ giải mã. Cụ thể, kiến trúc phần cứng có thể cập nhật đồng thời dữ liệu của z nút biến, cho phép tăng tốc độ thông lượng lên gấp nhiều lần.

2.2 Thuật Toán Giải Mã Min-Sum

Thuật toán giải mã Min-Sum là một phiên bản đơn giản hóa của thuật toán Belief Propagation (BP). Sự đơn giản hóa này được thực hiện bằng cách giảm thiểu tối đa độ phức tạp tính toán tại các khối chức năng chịu trách nhiệm cập nhật nút kiểm tra (Check Node Update - CNU). So với thuật toán BP gốc, việc ứng dụng thuật toán Min-Sum giúp giảm thiểu đáng kể tài nguyên phần cứng tiêu thụ cũng như làm giảm độ phức tạp xử lý toán học của khối CNU [7] mà vẫn duy trì được hiệu suất sửa lỗi ổn định.

Hệ thống giải mã quy ước các đại lượng toán học cơ bản bao gồm:

- c_n : Biểu thị cho bit thứ n của một từ mã phát đi từ nguồn.
- y_n : Giá trị tín hiệu thu được từ kênh truyền ở đầu vào bộ giải mã.

- $r_{mn}[k]$: Thông điệp truyền từ nút kiểm tra m đến nút biến n tại vòng lặp thứ k (thông điệp Check-to-Variable - CTV).
- $q_{mn}[k]$: Thông điệp truyền từ nút biến n đến nút kiểm tra m tại vòng lặp thứ k (thông điệp Variable-to-Check - VTC).
- $N(m)$: Tập hợp các nút biến tham gia vào phương trình kiểm tra của nút kiểm tra m .
- $M(n)$: Tập hợp các nút kiểm tra liên kết trực tiếp với nút biến n .
- $N(m) \setminus n$: Tập hợp các nút biến thuộc $N(m)$ ngoại trừ chính nút biến vị trí n .
- $M(n) \setminus m$: Tập hợp các nút kiểm tra thuộc $M(n)$ ngoại trừ chính nút kiểm tra vị trí m .

Thuật toán giải mã Min-Sum được vận hành tuần tự qua hai giai đoạn chính:

Bước 1: Khởi tạo (Initialization). Tại bước này, giá trị xác suất kênh truyền p_n (thông tin nội tại) của nút biến n được xác định dựa trên tỷ số của xác suất tiên nghiệm:

$$p_n = \log \frac{P(y_n | c_n = 0)}{P(y_n | c_n = 1)} \quad (2.1)$$

Đồng thời, giá trị ban đầu của các thông điệp CTV được thiết lập bằng không: $r_{mn}[0] = 0$.

Bước 2: Giải mã lặp (Iterative Decoding). Tại vòng lặp thứ k , thông điệp VTC là $q_{mn}[k]$ được tổng hợp bằng cách cộng thông tin nội tại của kênh với tổng các thông điệp phản hồi từ các nút kiểm tra liên kết còn lại thông qua công thức:

$$q_{mn}[k] = p_n + \sum_{m' \in \{M(n) \setminus m\}} r_{m'n}[k-1] \quad (2.2)$$

Bộ giải mã tiến hành thực hiện phép quyết định cứng bằng cách tính toán giá trị xác suất hậu nghiệm (A-Posterior Probability - APP) $\Lambda_n[k]$ tại nút biến n hiện tại:

$$\Lambda_n[k] = p_n + \sum_{m' \in M(n)} r_{m'n}[k-1] \quad (2.3)$$

Giá trị của bit kết quả x_n thuộc vectơ từ mã ước lượng sẽ được xác định theo quy tắc:

$$x_n = \begin{cases} 0 & \text{nếu } \Lambda_n[k] > 0 \\ 1 & \text{ngược lại} \end{cases} \quad (2.4)$$

Vòng lặp giải mã sẽ kết thúc hoàn toàn khi vectơ kết quả $x = [x_1, x_2, \dots, x_N]$ thỏa mãn đồng thời tất cả các phương trình kiểm tra chẵn lẻ mang tính tuyến tính của ma trận:

$$H \cdot x = 0 \quad (2.5)$$

Hoặc tiến trình lặp cũng sẽ tự động dừng lại nếu số vòng lặp hiện tại đạt đến giá trị ngưỡng tối đa được thiết lập từ trước trong cấu hình phần cứng hệ thống. Nếu từ mã hợp lệ, chế độ xuất kết quả được kích hoạt để đưa dữ liệu đã sửa lỗi ra ngoài đầu ra.

Nếu phép toán kiểm tra ma trận không thỏa mãn ($H \cdot x \neq 0$), thông điệp kiểm tra phản hồi ngược lại $r_{mn}[k]$ gửi đến nút biến m sẽ được tính toán dựa trên tích các dấu và giá trị cực tiểu của các thông điệp VTC thuộc tập liên kết loại trừ:

$$r_{mn}[k] = \left(\prod_{n' \in \{N(m) \setminus n\}} \text{sign}(q_{mn'}[k]) \right) \times \left(\alpha \times \min_{n' \in \{N(m) \setminus n\}} \{|q_{mn'}[k]|\} \right) \quad (2.6)$$

Trong đó, α là hệ số tỷ lệ điều chỉnh cố định ($\alpha \leq 1$) nhằm bù đắp cho sự suy giảm biên độ do việc đơn giản hóa từ BP sang Min-Sum.

2.2.1 Biểu thức rút gọn phục vụ tối ưu hóa phần cứng

Để tối ưu hóa kiến trúc đọc/ghi song song dữ liệu từ bộ nhớ cổng kép (Dual-port BRAM) trên FPGA, cấu trúc toán học của thuật toán được tinh chỉnh lại. Các phép tính được đưa về quan hệ trực tiếp giữa giá trị APP tổng thể và các thông điệp cập nhật nhằm giảm độ trễ logic.

- Thay vì thực hiện phép cộng tích lũy, VTC được trích xuất trực tiếp bằng cách trừ thông điệp CTV của vòng trước khỏi APP hiện tại:

$$VTC_n = APP_n - CTV_n, \quad n = 1 \dots d_r \quad (2.7)$$

với d_r là trọng số hàng của ma trận kiểm tra.

- Tính toán CTV mới:

$$CTV_{new_n} = \prod_{k=1}^N \text{sign}(VTC_k) \times \text{sign}(VTC_n) \times \min(|VTC|) \quad (2.8)$$

Biểu thức này được hiện thực hóa bằng mạng so sánh logic kết hợp với các cổng logic XOR để xử lý dấu.

- Cập nhật giá trị APP mới cho vòng lặp kế tiếp:

$$APP_{new_n} = VTC_n + CTV_{new_n}, \quad n = 1 \dots d_r \quad (2.9)$$

Kết quả APP mới này sẽ được ghi đè trở lại BRAM, chuẩn bị cho chu kỳ giải mã tiếp theo.

2.2.2 Ví dụ tổng quát về thuật toán Min-Sum

Giả sử ta có từ mã $c = [0 \ 0 \ 1 \ 0 \ 1 \ 1]$, được truyền qua kênh truyền với xác suất lai chéo $p = 0.2$ và giá trị tín hiệu thu được từ kênh truyền $y = [1 \ 0 \ 1 \ 0 \ 1 \ 1]$. [8]

Đầu tiên, ta tiến hành thiết lập ma trận kiểm tra chẵn lẻ H với kích thước 4×6 (4 phương trình kiểm tra và 6 bit từ mã) để tiện quan sát:

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} \quad (2.10)$$

Từ 2.1 ta có giá trị xác suất kênh truyền như sau:

$$p_n = \begin{cases} P(y_n|c_n = 0) = \log \frac{p}{1-p}, & \text{nếu } y_i = 1, \\ P(y_n|c_n = 1) = \log \frac{1-p}{p}, & \text{nếu } y_i = 0. \end{cases} \quad (2.11)$$

Như vậy đối với kênh truyền này ta lần lượt có giá trị như sau:

$$P(y_n|c_n = 0) = \log \frac{0.2}{1-0.2} = -1.3863, \quad P(y_n|c_n = 1) = \log \frac{1-0.2}{0.2} = 1.3863 \quad (2.12)$$

Từ đó ta có dãy Log-Likelihood Ratio (LLR) thu được từ kênh truyền như sau:

$$p = [-1.3863, 1.3863, -1.3863, 1.3863, -1.3863, -1.3863] \quad (2.13)$$

Tại vòng lặp 0, các thông điệp VTC chưa qua xử lý, nên sẽ nhận chính giá trị LLR ban đầu này:

$$\begin{aligned} n = 1: & VTC_{1,1} = -1.3863, VTC_{1,3} = -1.3863 \\ n = 2: & VTC_{2,1} = 1.3863, VTC_{2,2} = 1.3863 \\ n = 3: & VTC_{3,2} = -1.3863, VTC_{3,4} = -1.3863 \\ n = 4: & VTC_{4,1} = 1.3863, VTC_{4,4} = 1.3863 \\ n = 5: & VTC_{5,2} = -1.3863, VTC_{5,3} = -1.3863 \\ n = 6: & VTC_{6,3} = -1.3863, VTC_{6,4} = -1.3863 \end{aligned}$$

Tiếp đến, ta cập nhật giá trị CTV thông qua công thức 2.8 ứng với 3 nút kiểm tra:

$$\begin{aligned} m = 1: & CTV_{1,1} = 1.3863, CTV_{1,2} = -1.3863, CTV_{1,4} = -1.3863 \\ m = 2: & CTV_{2,2} = 1.3863, CTV_{2,3} = -1.3863, CTV_{2,5} = -1.3863 \\ m = 3: & CTV_{3,1} = 1.3863, CTV_{3,5} = 1.3863, CTV_{3,6} = 1.3863 \\ m = 4: & CTV_{4,3} = -1.3863, CTV_{4,4} = 1.3863, CTV_{4,6} = -1.3863 \end{aligned}$$

Tiếp theo, ta tiến hành cập nhật giá trị APP mới thông qua 2.9 cho vòng lặp tiếp theo:

$$\begin{aligned} APP_1 &= p_1 + CTV_{1,1} + CTV_{3,1} = -1.3863 + 1.3863 + 1.3863 = 1.3863 \\ APP_2 &= p_2 + CTV_{1,2} + CTV_{2,2} = 1.3863 - 1.3863 + 1.3863 = 1.3863 \\ APP_3 &= p_3 + CTV_{2,3} + CTV_{4,3} = -1.3863 - 1.3863 - 1.3863 = -4.1589 \\ APP_4 &= p_4 + CTV_{1,4} + CTV_{4,4} = 1.3863 - 1.3863 + 1.3863 = 1.3863 \\ APP_5 &= p_5 + CTV_{2,5} + CTV_{3,5} = -1.3863 - 1.3863 + 1.3863 = -1.3863 \\ APP_6 &= p_6 + CTV_{3,6} + CTV_{4,6} = -1.3863 + 1.3863 - 1.3863 = -1.3863 \end{aligned}$$

Căn cứ vào dấu của bộ giá trị APP mới, hệ thống đưa ra phép quyết định cứng dựa vào quy tắc 2.4. Khi ấy ta có từ mã dự đoán:

$$x = [0 \ 0 \ 1 \ 0 \ 1 \ 1] \quad (2.14)$$

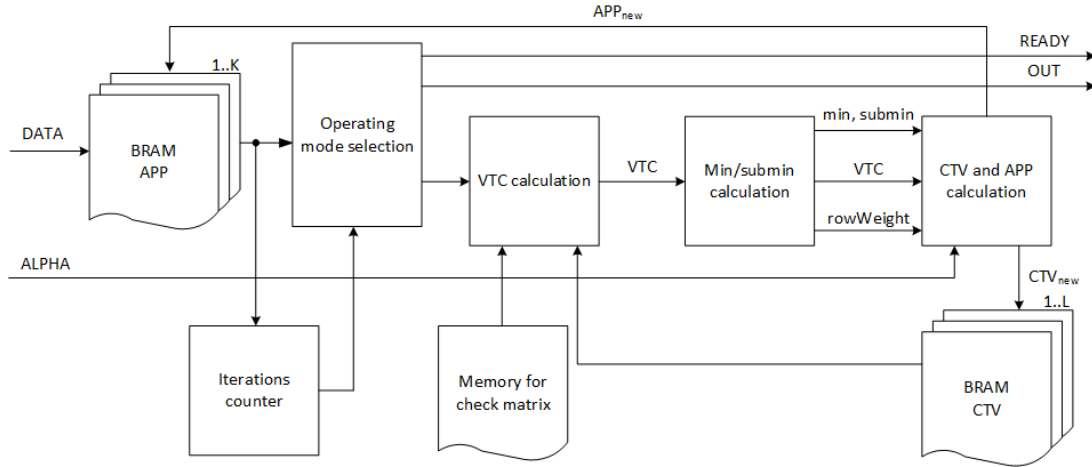
Để kiểm tra nếu từ mã x đã đúng hay chưa, ta áp dụng 2.5 và thu được:

$$\mathbf{x}H^T = [0 \ 0 \ 1 \ 0 \ 1 \ 1] \begin{bmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} = [0 \ 0 \ 0 \ 0] \quad (2.15)$$

Như vậy x là từ mã hợp lệ.

3. Thiết kế và thực hiện phần cứng

3.1 Tổng quan hoạt động hệ thống



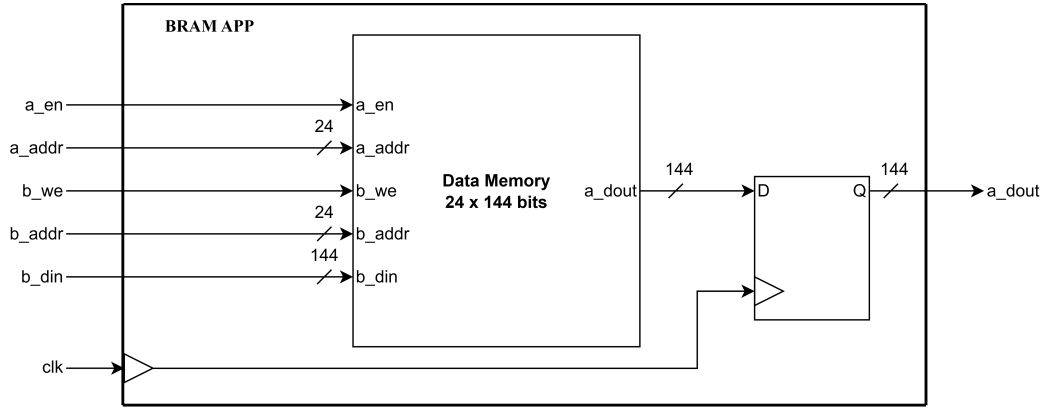
Hình 3.1: Kiến trúc bộ giải mã LDPC

Luồng dữ liệu bên trong bộ giải mã diễn ra theo một chu trình khép kín, được đồng bộ chặt chẽ qua từng vòng lặp:

- Khởi tạo: Dòng bit dữ liệu đầu vào dạng nối tiếp được chuyển đổi thành dạng song song và ghi vào khối bộ nhớ BRAM APP.
- Bắt đầu vòng lặp: Tại mỗi chu kỳ lặp, dữ liệu xác suất hậu nghiệm APP được đọc ra từ BRAM APP. Đồng thời, các thông tin về vị trí cấu trúc mạng được truy xuất từ bộ nhớ lưu trữ ma trận kiểm tra để định tuyến chính xác dữ liệu.
- Tính toán VTC: Khối VTC calculation nhận APP và các giá trị CTV từ vòng lặp trước (lưu trong BRAM CTV), thực hiện phép trừ để tạo ra thông điệp VTC gửi đến các nút kiểm tra.
- Tìm cực tiểu (Min/submin): Các giá trị VTC này được đẩy vào khối Min/submin calculation để tách biệt phần dấu và phần độ lớn, sau đó chạy qua một mạng lưới mạch so sánh nhằm tìm ra hai giá trị có độ lớn nhỏ nhất và nhỏ thứ hai của mỗi hàng.
- Tính toán cập nhật: Khối CTV and APP calculation sử dụng các giá trị cực tiểu này, kết hợp với phép XOR các bit dấu và hệ số tỉ lệ α để tính ra thông điệp CTV mới (CTV_{new}). Ngay lập tức, khối này cộng ngược VTC với CTV_{new} để tạo ra xác suất hậu nghiệm mới (APP_{new}).
- Phản hồi và xuất dữ liệu: APP_{new} được ghi đè trở lại BRAM APP và CTV_{new} được ghi vào BRAM CTV cho vòng lặp tiếp theo. Nếu bộ đếm xác định đã đạt số vòng lặp tối đa, luồng dữ liệu phản hồi bị ngắt, tín hiệu READY được kích hoạt và từ mã đã được giải mã sẽ được đẩy ra đường OUT.

3.2 Khối BRAM

3.2.1 Khối BRAM APP



Hình 3.2: Kiến trúc khối BRAM APP

Về mặt yêu cầu thiết kế, khối BRAM APP đóng vai trò chứa các giá trị APP trong suốt quá trình lập của thuật toán Min-Sum. Các giá trị này ban đầu chính là các quyết định mềm thu được từ bộ giải điều chế của kênh truyền. Nhằm giải quyết bài toán về băng thông dữ liệu trong các thiết kế giải mã tốc độ cao, bộ nhớ này bắt buộc phải được thiết kế dưới dạng bộ nhớ RAM cổng kép (dual-port). Điều này cho phép phần cứng có thể đọc và ghi các giá trị một cách độc lập trong cùng một chu kỳ xung nhịp.

Kiến trúc tổng quan và các thông số kỹ thuật chi tiết bao gồm:

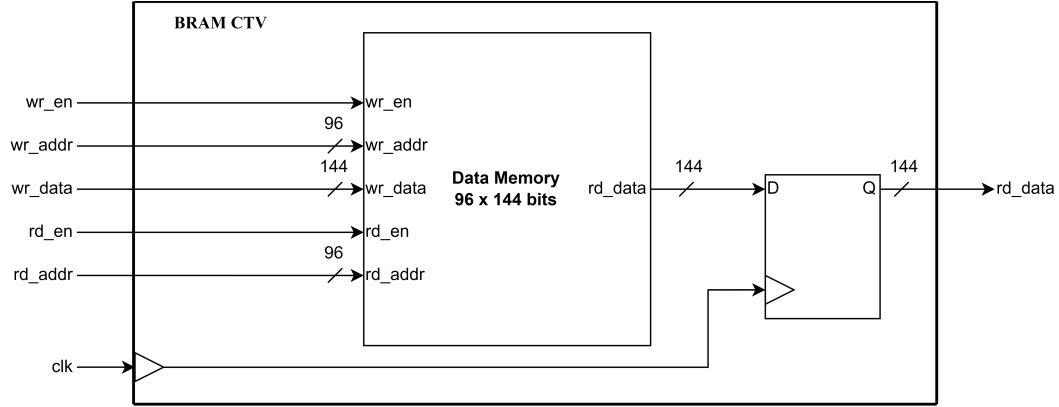
- Mã nguồn định nghĩa sẵn hai tham số cơ bản là độ sâu bộ nhớ $DEPTH = 24$ và độ rộng dữ liệu $WIDTH = 144$ bit. Không gian lưu trữ thực tế là một mảng thanh ghi hai chiều mang tên *mem*.
- Để hỗ trợ việc đọc và ghi song song các giá trị APP, tổng thể bộ nhớ không nằm ở một khối duy nhất mà được chia nhỏ thành $K = N/z$ khối. Trong đó, N là chiều dài của từ mã và z là kích thước của ma trận con tuần hoàn.
- Kích thước của mỗi khối BRAM APP phụ thuộc trực tiếp vào số lượng ký hiệu trong ma trận con tuần hoàn z và độ rộng của dữ liệu đầu vào. Cụ thể, dung lượng được cấp phát tính bằng $(z \times \text{weight_of_input_data}) \times (N/z)$.
- Giao tiếp Cổng A (Cổng Đọc): Cổng này chịu sự điều khiển của tín hiệu cho phép đọc a_en . Dữ liệu đầu ra a_dout được trích xuất từ vị trí a_addr . Giao tiếp Cổng B (Cổng Ghi): Cổng này nhận tín hiệu cho phép ghi b_we để nạp dữ liệu b_din vào địa chỉ b_addr .

Nguyên lý hoạt động của khối BRAM APP diễn ra như sau:

- Trong pha giải mã: Tại mỗi cạnh lên của xung nhịp clk , hệ thống chỉ trích xuất một phần các khối bộ nhớ BRAM APP thông qua Cổng A. Việc khối nào được đọc hoàn toàn phụ thuộc vào cấu trúc của ma trận kiểm tra chẵn lẻ LDPC. Dữ liệu này được đẩy vào khối "VTC calculation". Đồng thời, các giá trị APP cập nhật (APP_{new}) được tính toán xong sẽ ngay lập tức được ghi đè ngược lại vào bộ nhớ thông qua Cổng B.
- Trong pha xuất kết quả: Khi khối "Iterations counter" báo hiệu đã hoàn thành số vòng lặp tối đa, khối "Operating mode selection" sẽ chuyển trạng thái hệ thống. Lúc

này, toàn bộ các khối của bộ nhớ BRAM APP sẽ được đọc và đẩy thẳng ra ngõ ra *READY OUT* để cung cấp từ mã đã được giải mã hoàn chỉnh.

3.2.2 Khối BRAM CTV



Hình 3.3: Kiến trúc khối BRAM CTV

Về mặt yêu cầu thiết kế, khối BRAM CTV là thành phần lưu trữ các bản tin CTV, tương đương với thông điệp $r_{mn}[k]$ truyền từ nút kiểm tra m đến nút biến n trong thuật toán đồ thị Tanner. Vì ma trận QC-LDPC của các chuẩn WiFi (IEEE 802.11n) và WiMAX (IEEE 802.16e) có tính chất tuần hoàn, kiến trúc bộ giải mã yêu cầu phải cập nhật đồng thời tất cả các giá trị APP cho một hàng hiện tại của ma trận. Do đó, hệ thống bắt buộc phải khởi tạo z phiên bản bộ nhớ CTV chạy hoàn toàn song song (với z là số nhân lỗi tương đương kích thước ma trận con).

Kiến trúc tổng quan và các thông số kỹ thuật chi tiết bao gồm:

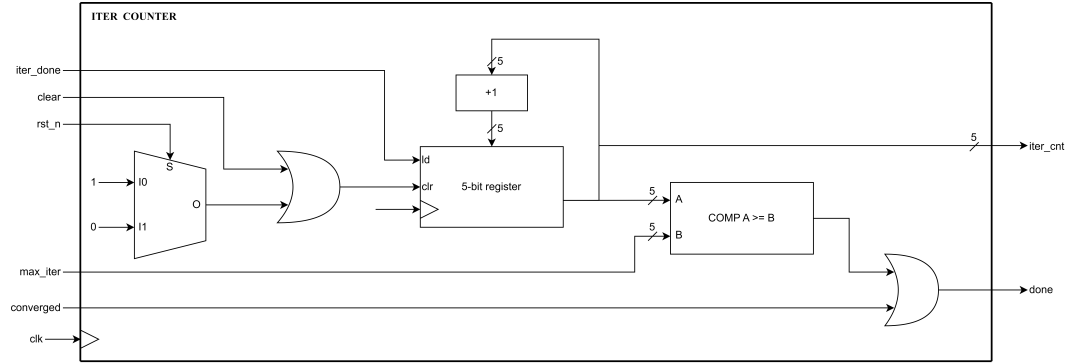
- Khác với BRAM APP, dung lượng của BRAM CTV được thiết kế để tránh lãng phí tài nguyên trên FPGA. Cụ thể, độ sâu của bộ nhớ được quy định bằng đúng số lượng hàng khối nhân với trọng số hàng tối đa là 96.
- Mã nguồn định nghĩa độ sâu mặc định $DEPTH = 96$ và độ rộng $WIDTH = 144$ bit. Tương đương với $z \times DW$, tức z lần dữ liệu song song, mỗi lần rộng DW bit.
- Kích thước chính xác của nhóm bộ nhớ này được ánh xạ theo cấu trúc ma trận LDPC với công thức: Trọng số hàng của ma trận $\times$ Kích thước dữ liệu đầu vào $\times$ Số lượng hàng của ma trận.
- Mảng thanh ghi mem được truy cập qua hai tiến trình phân cứng độc lập. Tiến trình đọc được kiểm soát bởi rd_en , rd_addr để xuất rd_data . Tiến trình ghi được kiểm soát bởi wr_en , wr_addr để nạp wr_data .

Nguyên lý hoạt động của khối BRAM CTV diễn ra như sau:

- Cung cấp dữ liệu (Pha Đọc): Khởi đầu mỗi chu kỳ lặp, các giá trị CTV ban đầu được thiết lập bằng 0, sau đó sẽ được xuất ra thông qua ngõ rd_data khi có tín hiệu rd_en tại cạnh lên của xung clk. Các giá trị CTV cũ này được đưa trực tiếp vào khối "VTC calculation" để thực hiện phép toán $VTC_n = APP_n - CTV_n$.
- Cập nhật dữ liệu (Pha Ghi): Tại cuối chu trình của một hàng, khối "CTV and APP calculation" sẽ tổng hợp các giá trị VTC cực tiểu min và cực tiểu phụ submin, sau đó khối sẽ xử lý dấu (Signs XORing) để tạo ra bản tin CTV_{new} . Ngay lập tức, khi có xung clk và tín hiệu wr_en tích cực, tập dữ liệu CTV_{new} này được nạp vào chân

wr_data và ghi đè vào địa chỉ wr_addr tương ứng. Quá trình này đảm bảo dữ liệu luôn sẵn sàng cho vòng lặp giải mã tiếp theo.

3.3 Khối Iterations Counter



Hình 3.4: Kiến trúc khối Iterations Counter

Về mặt yêu cầu thiết kế, khối iter counter là thành phần kiểm soát số vòng lặp giải mã, tương đương với biến đếm số lần lặp trong các thuật toán lặp. Vì thuật toán giải mã yêu cầu lặp đi lặp lại quá trình cập nhật thông điệp để hội tụ, kiến trúc bộ giải mã yêu cầu phải giới hạn số vòng lặp tối đa và có khả năng dừng sớm khi đã tìm ra từ mã đúng. Do đó, hệ thống bắt buộc phải khởi tạo bộ đếm này để theo dõi tiến trình và phát tín hiệu hoàn thành (*done*) chuyển khối "Operating mode selection" sang chế độ xuất kết quả.

Kiến trúc tổng quan và các thông số kỹ thuật chi tiết bao gồm:

- Khác với các khối lưu trữ dữ liệu, khối iter counter được thiết kế dưới dạng một bộ đếm đồng bộ kết hợp logic tổ hợp để tránh giải mã mắc kẹt trong vòng lặp vô tận.
- Mã nguồn định nghĩa độ rộng mặc định $ITER_W = 5$ bit và cờ cho phép dừng sớm $DONE = 1$. Tương đương với việc phần cứng có thể đếm và quản lý tối đa 32 vòng lặp, mỗi vòng được biểu diễn trên 5 bit dữ liệu.
- Giới hạn lặp chính xác của hệ thống được ánh xạ theo cấu hình động với đầu vào max_iter có độ rộng $ITER_W$ bit, quy định số vòng lặp tối đa cho từng phiên chạy.
- Khối đếm được cập nhật qua một tiến trình phần cứng tuần tự độc lập. Tiến trình này được kiểm soát bởi tín hiệu đồng hồ clk và tín hiệu reset không đồng bộ rst_n để xuất ra giá trị vòng lặp hiện tại $iter_cnt$ và cờ trạng thái done.

Nguyên lý hoạt động của khối iter counter diễn ra như sau:

- Khởi tạo và làm mới dữ liệu (Pha Khởi tạo): Khởi đầu hệ thống (tín hiệu rst_n ở mức thấp) hoặc khi hệ thống bắt đầu xử lý một khung giải mã mới (tín hiệu clear tích cực), các giá trị của bộ đếm $iter_cnt$ và cờ $done$ ban đầu được thiết lập bằng 0.
- Cập nhật và đánh giá (Pha Đếm): Tại quá trình giải mã, khi có xung $iter_done$ báo hiệu kết thúc một vòng lặp, khối sẽ thực hiện phép toán $iter_cnt + 1$ để tăng biến đếm. Ngay lập tức, khối sẽ tổng hợp các điều kiện để xét duyệt hoàn thành: nếu giá trị lặp tiếp theo đạt tới ngưỡng max_iter hoặc nếu cờ $DONE$ tích cực đi kèm với việc bật sáng sớm thì tín hiệu $done$ sẽ được nạp mức cao. Quá trình này

đảm bảo tín hiệu hoàn thành luôn sẵn sàng để hệ thống dừng giải mã đúng thời điểm.

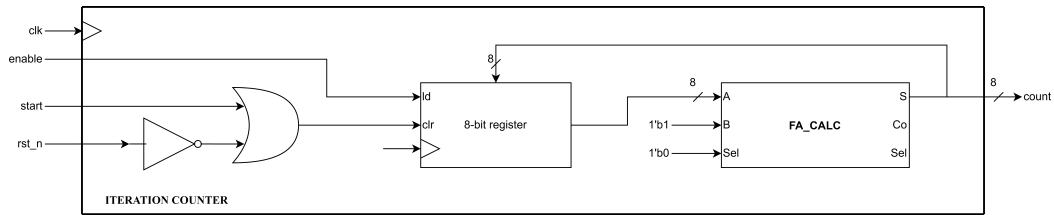
3.4 Khối Operating Mode Selection

Trong hệ thống bộ giải mã LDPC, quá trình giải mã thông thường sẽ kết thúc khi toàn bộ từ mã thỏa mãn phương trình kiểm tra chẵn lẻ $Hx = 0$, hoặc khi mạch đếm vòng lặp đạt đến giới hạn số vòng lặp tối đa được thiết lập từ trước. Dựa trên nguyên lý đó, khối **mode_select_top** được thiết kế theo cấu trúc phân rã nhằm quản lý việc chuyển đổi trạng thái giữa quá trình lặp và quá trình xuất dữ liệu. Thiết kế này tách biệt rõ ràng luồng điều khiển (đếm vòng lặp, kiểm tra điều kiện dừng) và luồng định tuyến dữ liệu thông qua 5 module con hoạt động phối hợp như sau:

Khối đếm vòng lặp (iteration_counter): Đây là một mạch tuần tự chịu trách nhiệm theo dõi số lượng vòng lặp hiện tại của bộ giải mã. Thay vì sử dụng toán tử cộng (+) thông thường, module này tái sử dụng khối tính toán toán học lõi **fa_calc** với chân chọn chế độ $Sel = 0$ (tương đương phép cộng) nhằm đồng bộ hóa việc sử dụng tài nguyên với các module khác trong hệ thống. Khi tín hiệu **enable** ở mức cao, giá trị bộ đếm được cập nhật:

$$count_plus_one = count + 1 \quad (3.1)$$

Bên cạnh đó, mạch được trang bị cờ **start** kết hợp với reset bất đồng bộ (**reset_n**), cho phép hệ thống chủ động khởi tạo lại số đếm về 0 khi bắt đầu xử lý một khung dữ liệu mới.

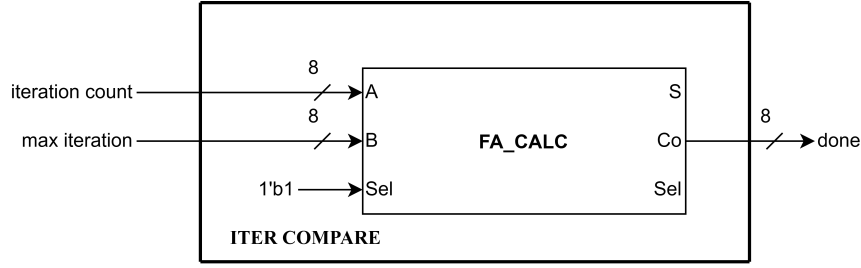


Hình 3.5: Kiến trúc khối Iterations Counter

Khối so sánh điều kiện dừng (iter_compare): Mô-đun này có nhiệm vụ phát hiện thời điểm bộ giải mã đã đạt đến ngưỡng $max_iteration$. Để tối ưu hóa cổng logic, mạch tiếp tục sử dụng khối **fa_calc** nhưng được cấu hình như một bộ trừ ($Sel = 1$). Bằng cách thực hiện phép trừ giữa số vòng lặp hiện tại và giới hạn tối đa, tín hiệu hoàn tất (**done**) được trích xuất trực tiếp thông qua cờ nhớ C_o :

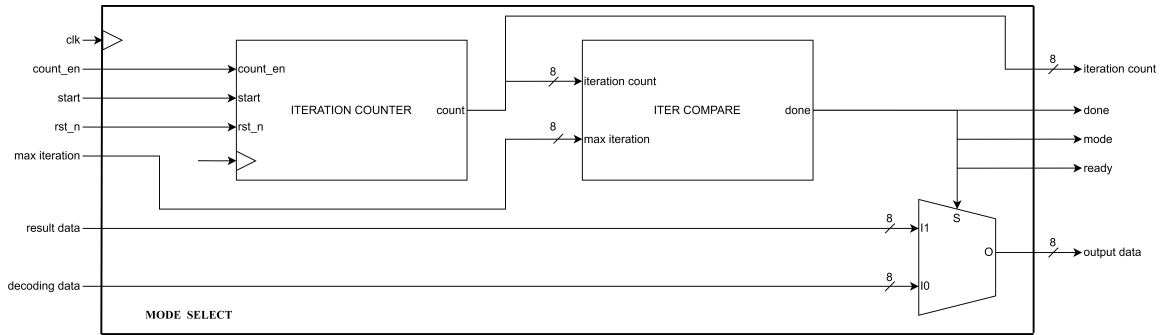
$$done = C_o(iteration_count - max_iteration) \quad (3.2)$$

Ngay khi $iteration_count$ bằng hoặc vượt mức $max_iteration$, cờ tràn C_o sẽ lên mức 1, báo hiệu hệ thống cần kết thúc quá trình lặp.



Hình 3.6: Kiến trúc khối Iterations Compare

Khối giao diện mức cao nhất (mode_select_top): Module mode_select_top thực hiện chức năng wrapper. Việc đóng gói khối dây (iteration_counter) cùng với các khối tổ hợp (iter_compare) thành một IP duy nhất giúp thiết lập ranh giới rõ ràng cho kiến trúc. Điều này tạo thuận lợi cho việc synthesis, đồng thời giúp công cụ thiết kế dễ dàng đánh giá timing analysis của toàn bộ module mà không bị nhiễu bởi các khối bên ngoài.



Hình 3.7: Kiến trúc khối Mode Select

Bộ định tuyến dữ liệu: Đóng vai trò là một bộ ghép kênh 2-to-1 thuần tổ hợp. Chức năng chính của bộ này là lựa chọn nguồn dữ liệu ngõ ra dựa vào tín hiệu cờ trạng thái mode:

$$output_data = \begin{cases} result_data & \text{nếu } mode = 1 \\ decoding_data & \text{nếu } mode = 0 \end{cases} \quad (3.3)$$

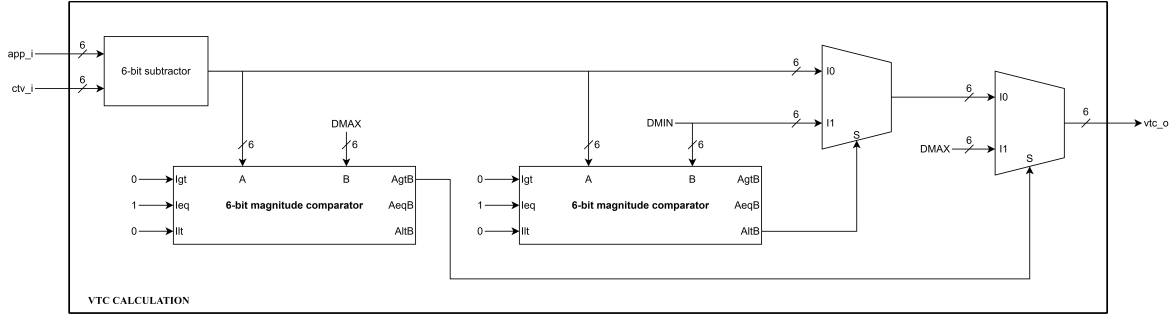
Kiến trúc này đảm bảo mạch luôn giữ ổn định luồng dữ liệu trung gian (decoding_data) trong suốt quá trình giải mã, và chỉ chuyển qua luồng đầu ra cuối cùng (result_data) khi hệ thống nhận được lệnh kết thúc.

Bộ quản lý trạng thái trung gian: Đây là cầu nối đảm nhận vai trò liên kết logic điều khiển (iter_compare) và phần tử định tuyến datapath (mode_mux). Nó tiếp nhận cờ done từ bộ so sánh và phân phối thành các tín hiệu điều khiển toàn cục, bao gồm mode và ready dùng để thông báo cho các khối xử lý ở tầng cao hơn về trạng thái dữ liệu:

$$mode = ready = done \quad (3.4)$$

3.5 Khối VTC Calculation

Khối VTC calculation chịu trách nhiệm tính toán các thông điệp VTC trong cấu trúc của bộ giải mã LDPC. Khối này nhận đầu vào là các giá trị xác suất hậu nghiệm APP từ khối bộ nhớ BRAM APP. Dựa trên APP và giá trị thông điệp CTV tương ứng, khối sẽ



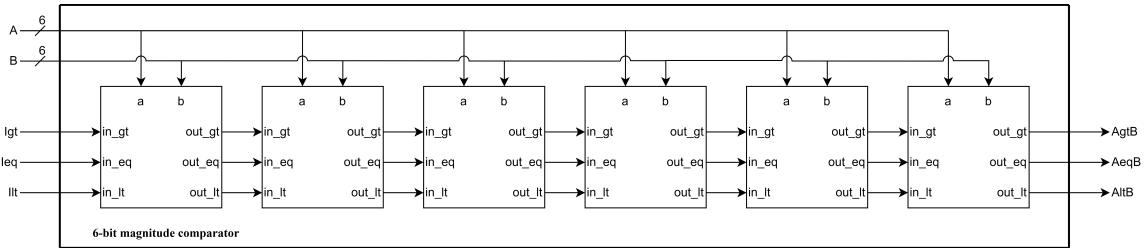
Hình 3.8: Kiến trúc khối VTC Calculation

tính toán ra tín hiệu VTC. Đầu ra VTC sau đó sẽ được chuyển tiếp đến khối tính toán Min/submin để thực hiện các bước tiếp theo của thuật toán giải mã Min-Sum. Khối được thiết kế để xử lý song song, trong đó số lượng bộ xử lý tương đương với kích thước ma trận con Z , cho phép cập nhật đồng thời tất cả các giá trị APP được sử dụng bởi hàng hiện tại của ma trận kiểm tra. Nguyên lý hoạt động của khối dựa trên việc hiện thực hóa công thức tính toán bản tin VTC 2.7.

Trong phần cứng, các bản tin APP được biểu diễn dưới dạng số nguyên có dấu với độ rộng $DW = 8$ bit. Phép trừ trực tiếp giữa hai số có dấu dễ dẫn đến hiện tượng tràn số, khiến một giá trị mang độ tin cậy cao bị đảo dấu thành độ tin cậy thấp. Để khắc phục, thiết kế tích hợp mạch trừ bão hòa kẹp giá trị ngõ ra trong dải $[-128, 127]$ theo logic:

$$VTC_o[i] = \begin{cases} D_{MAX}, & \text{nếu } (APP_i - CTV_i) > D_{MAX} \\ D_{MIN}, & \text{nếu } (APP_i - CTV_i) < D_{MIN} \\ APP_i - CTV_i, & \text{ngược lại} \end{cases} \quad (3.5)$$

Đối với trường hợp thiết kế tham số $DW = 8$ bit, hai ngưỡng giới hạn bão hòa được xác định là $D_{MAX} = +127$ và $D_{MIN} = -128$.



Hình 3.9: Kiến trúc Bộ so sánh độ lớn số 6 bit

Kiến trúc phần cứng mức logic của khối vtc_calc được hiện thực hóa thông qua các thành phần liên kết chặt chẽ:

- **Bộ trừ có dấu 6-bit:** Thực hiện phép trừ giữa hai lần dữ liệu ngõ vào $diff = APP_i - CTV_i$. Giá trị hiệu số $diff$ được mở rộng dải bit trung gian nhằm đảm bảo không mất thông tin bit dấu khi xảy ra tràn số.
- **Chuỗi bộ so sánh độ lớn 6-bit (6-bit Magnitude Comparator):** Tín hiệu hiệu số $diff$ được đưa song song vào hai bộ so sánh 6-bit để kiểm tra điều kiện bão hòa:
 - Bộ so sánh thứ nhất đối chiếu $diff$ với ngưỡng cực đại $D_{MAX} = +31$.

- Bộ so sánh thứ hai đối chiếu $diff$ với ngưỡng cực tiểu $D_{MIN} = -32$.

Mỗi bộ so sánh độ lớn 6-bit được cấu tạo từ một chuỗi nối tầng của 6 ô so sánh 1-bit từ bit có trọng số cao nhất MSB đến bit thấp nhất LSB. Tại mỗi bit i , ô so sánh tiếp nhận hai bit dữ liệu đầu vào a_i, b_i cùng các cờ trạng thái từ tầng trước là in_gt, in_eq, in_lt để cập nhật cờ ngõ ra out_gt, out_eq, out_lt theo các biểu thức Boolean:

$$out_gt = in_gt \vee (in_eq \wedge a_i \wedge \bar{b}_i) \quad (3.6)$$

$$out_eq = in_eq \wedge (a_i \oplus b_i) \quad (3.7)$$

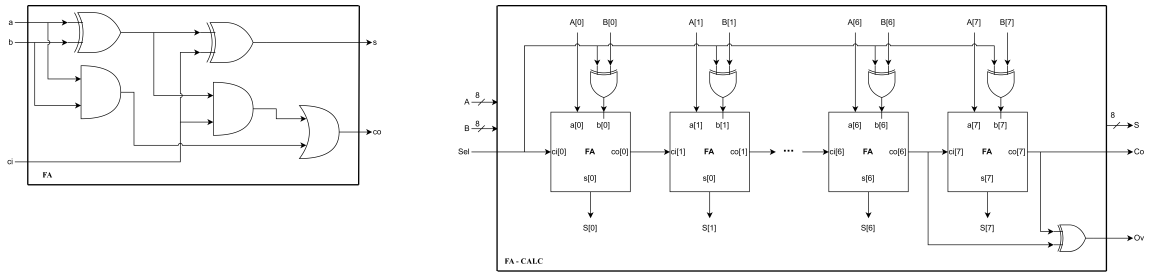
$$out_lt = in_lt \vee (in_eq \wedge \bar{a}_i \wedge b_i) \quad (3.8)$$

- **Bộ dồn kênh MUX:** Tín hiệu điều khiển ngõ ra $AgtB$ và $AltB$ từ các bộ so sánh 6-bit sẽ điều khiển mạng MUX chọn lựa giá trị xuất ra VTC_o . Nếu phát hiện tràn trên ($diff > D_{MAX}$), MUX chuyển ngõ ra về D_{MAX} ; nếu tràn dưới ($diff < D_{MIN}$), ngõ ra chọn D_{MIN} ; trường hợp không tràn, giá trị hiệu $diff$ ban đầu được truyền trực tiếp ra ngõ ra.

3.6 Khối Min/Submin Calculation

Khối `min_submin_calc` đóng vai trò là lõi toán học trung tâm của quá trình cập nhật nút kiểm tra trong thuật toán MSA của bộ giải mã LDPC. Đối với một hàng có trọng số $d_r = 7$ (nhận 7 bản tin đầu vào), chức năng cốt lõi của khối là xử lý tập hợp các bản tin VTC để tạo ra các bản tin CTV phản hồi.

Để tối ưu hóa tài nguyên và tăng cường hiệu năng trên FPGA/ASIC, thiết kế phần cứng của khối xử lý nút kiểm tra này được phân rã thành các module con hoạt động theo chuỗi dữ liệu, được xây dựng từ các khối tính toán cơ sở bao gồm bộ cộng toàn phần `fa`, bộ toán học `fa_calc`, và bộ so sánh `comp_min`.



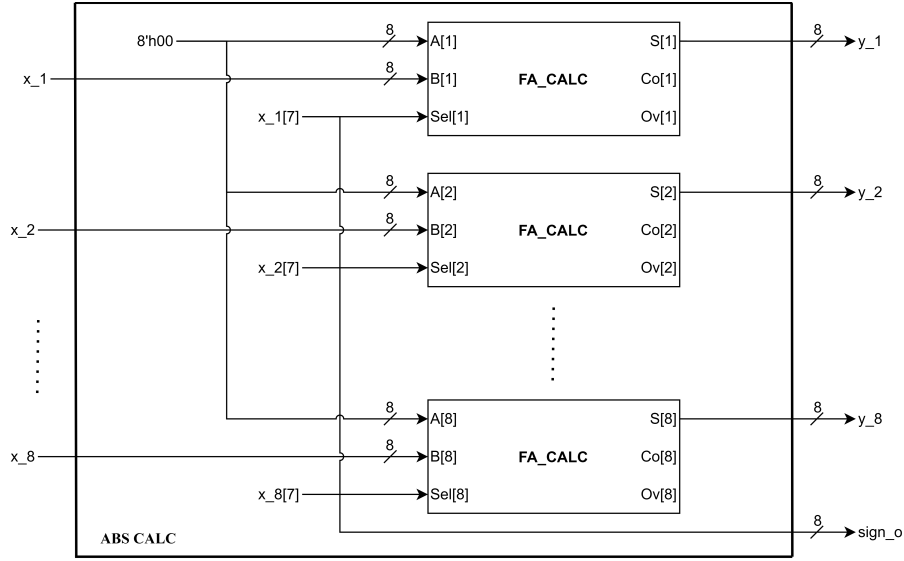
Hình 3.10: Kiến trúc phần tử con full adder 8 bit

Khối tách dấu và biên độ (`abs_calc`): Bước đầu tiên của thuật toán Min-Sum yêu cầu tách bản tin VTC (biểu diễn dưới dạng số có dấu bù 2 - 8 bit) thành hai thành phần độc lập: bit dấu ($sign$) và biên độ ($magnitude$). Khối này sử dụng các bộ tính toán `fa_calc` (dựa trên chuỗi nối tiếp các bộ cộng toàn phần `fa`) để thực hiện việc chuyển đổi bù 2 nếu dữ liệu là số âm. Với tín hiệu chọn Sel được gán bằng bit có trọng số cao nhất (MSB), biên độ y_i và bit dấu $sign_o_i$ được xác định theo biểu thức:

$$sign_o_i = x_i[7]$$

$$y_i = \begin{cases} x_i & \text{nếu } sign_o_i = 0 \\ \sim x_i + 1 & \text{nếu } sign_o_i = 1 \end{cases} \quad (3.9)$$

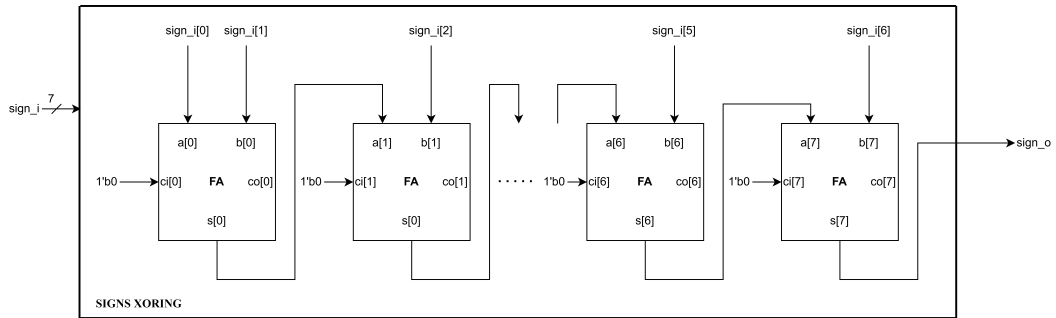
(Lưu ý: Phép toán $\sim x_i + 1$ được thực hiện tối ưu bằng cách đưa tín hiệu MSB vào chân Carry-in (C_i) và cổng XOR của *fa_calc*).



Hình 3.11: Kiến trúc Khối ABS

Khối tính toán bit dấu tổng (signs_xoring): Đồng thời với quá trình xử lý biên độ, các bit dấu của 7 bản tin đầu vào được đưa qua một chuỗi các cổng XOR để tìm ra bit dấu tích lũy toàn cục (*sign_total*). Điểm đặc biệt trong thiết kế phần cứng ở đây là việc tái sử dụng cấu trúc bộ cộng toàn phần (*fa*) với chân $C_{in} = 0$, biến chân *Sum* thành cổng XOR 2 ngõ vào để tạo thành một chuỗi XOR liên tiếp:

$$sign_total = \bigoplus_{i=1}^7 sign_o_i \quad (3.10)$$



Hình 3.12: Kiến trúc Khối SignsXoring

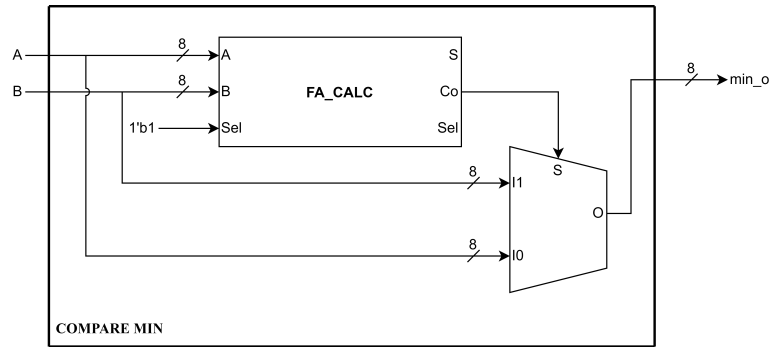
Cây so sánh tìm biên độ cực tiểu (comp_tree & comp_min): Đây là thành phần đóng vai trò tìm giá trị biên độ nhỏ nhất (Min) và nhỏ nhì (Submin) để phân phối lại cho các ngõ ra. Thay vì duyệt tuần tự, thiết kế sử dụng cấu trúc cây so sánh được xây dựng từ 18 bộ *comp_min*.

Bộ *comp_min* thực hiện phép trừ $A - B$ thông qua *fa_calc* (với *Sel* = 1). Dựa vào cờ tràn C_o , phần cứng sẽ quyết định giá trị nhỏ hơn. Cây so sánh này không chỉ tìm ra giá

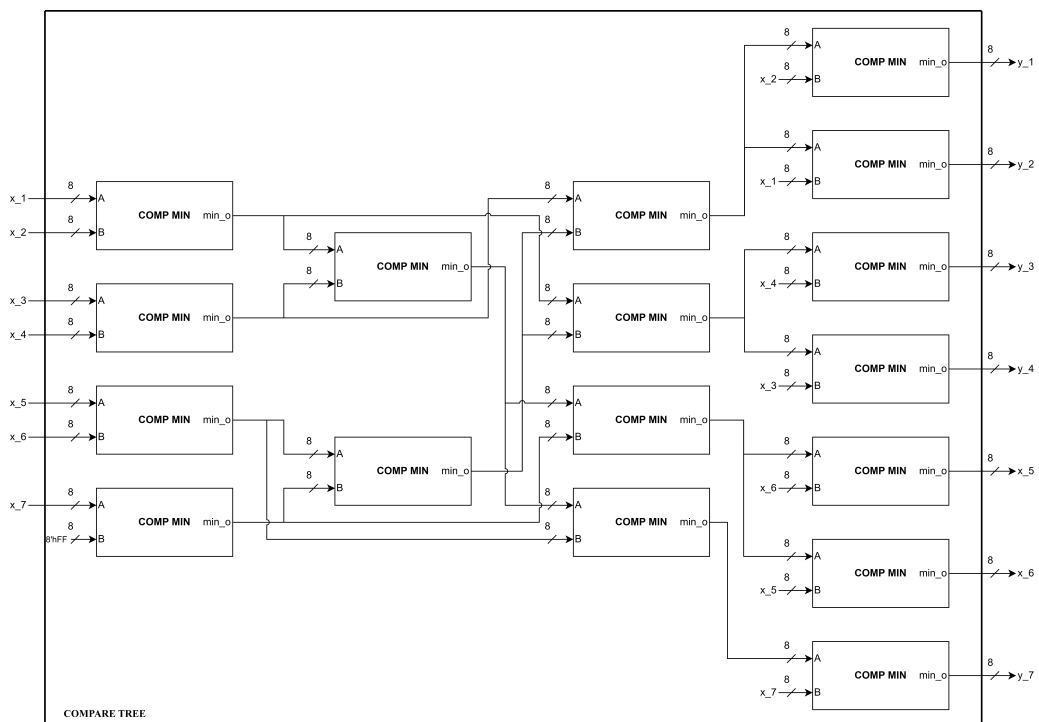
trị cực tiểu toàn cục (min_all), mà còn tính toán trước giá trị biên độ sẽ được gửi trả về cho từng nhánh thứ i (gọi là y_i) theo nguyên tắc loại trừ chính nó:

$$y_i = \min_{j \in \{1..7\}, j \neq i} (|VTC_j|) \quad (3.11)$$

Giá trị ngõ ra y_i sẽ chính là cực tiểu thứ nhất (min_1) nếu ngõ vào x_i không phải là cực tiểu thứ nhất, và sẽ là cực tiểu thứ hai (min_2 hay submin) nếu x_i chính là giá trị cực tiểu thứ nhất.



Hình 3.13: Kiến trúc phần tử con Comp min



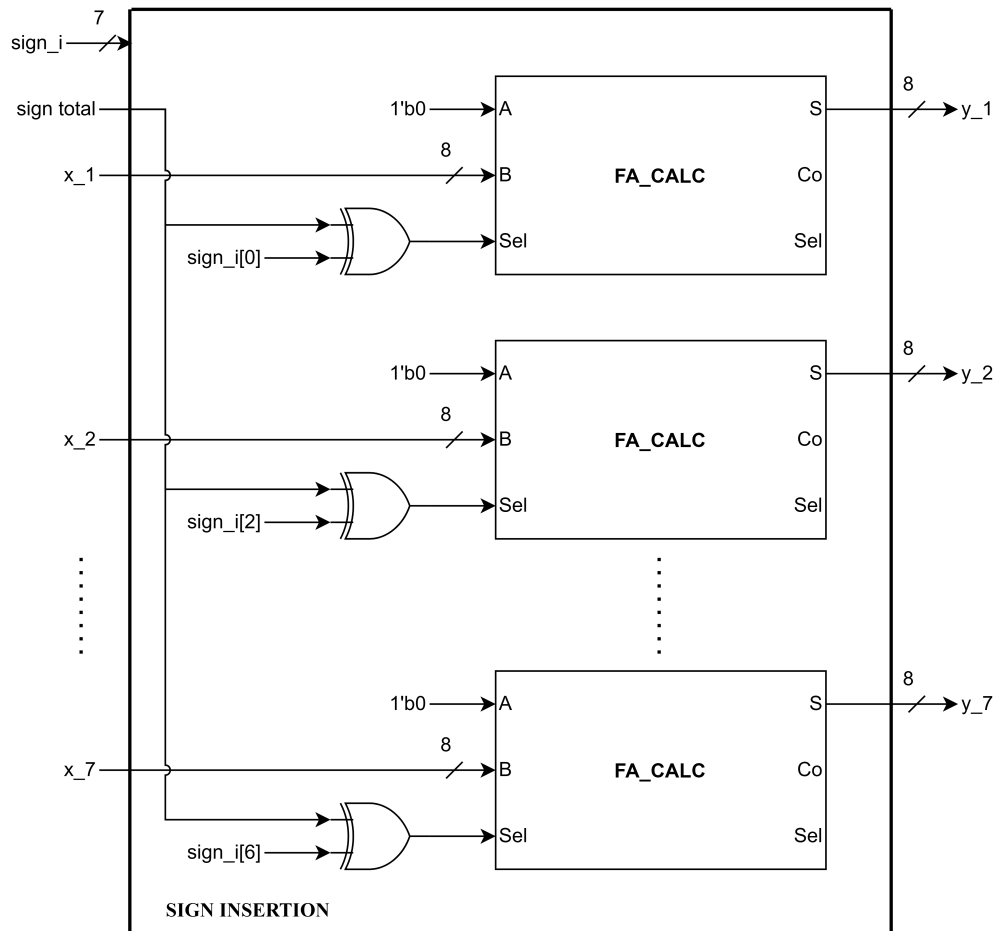
Hình 3.14: Kiến trúc Khối Compare tree

Khôi khôi phục dấu và tạo bản tin CTV (sign_insertion): Bước cuối cùng của chuỗi xử lý là kết hợp bit dấu mới và biên độ để tạo thành bản tin hoàn chỉnh dạng bù 2. Bit dấu cho bản tin CTV_i trên nhánh thứ i được cập nhật bằng cách XOR bit dấu toàn cục với bit dấu ban đầu của chính nó. Sau đó, dựa vào bit dấu mới này, dữ liệu lại được

đưa qua khối `fa_calc` để chuyển đổi từ biên độ thuần túy sang số có dấu bù 2:

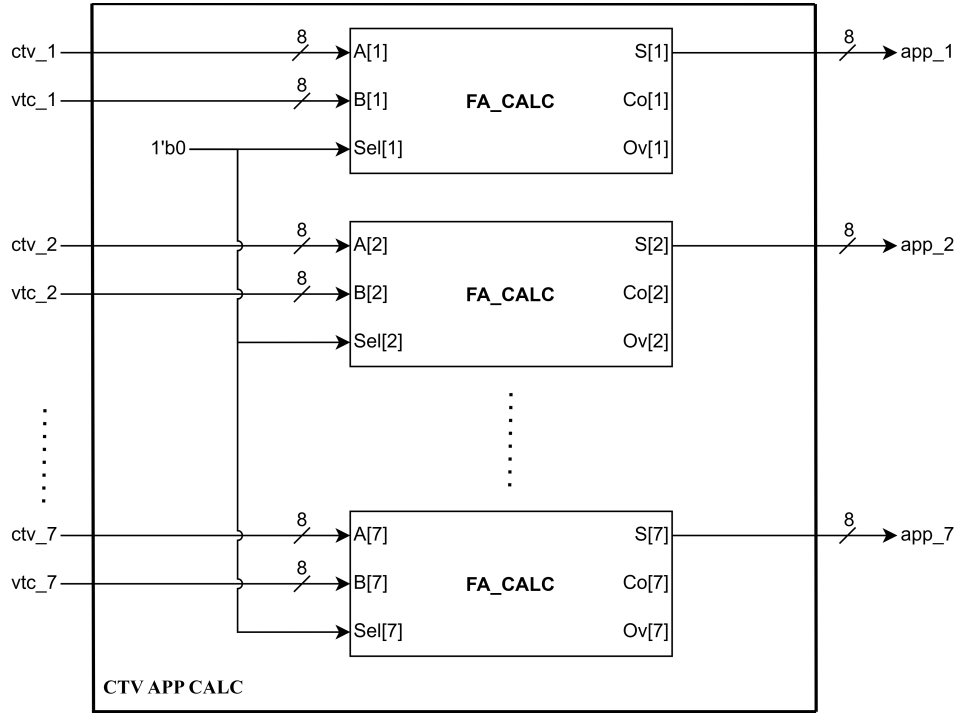
$$\begin{aligned} Sign_{CTV_i} &= sign_total \oplus sign_o_i \\ CTV_i &= \begin{cases} y_i & \text{nếu } Sign_{CTV_i} = 0 \\ \sim y_i + 1 & \text{nếu } Sign_{CTV_i} = 1 \end{cases} \end{aligned} \quad (3.12)$$

Nhìn chung, bằng cách sử dụng chung một mô-đun toán học lõi (`fa_calc`) cho cả quá trình lấy trị tuyệt đối, so sánh trừ, và khôi phục dấu, kiến trúc này giúp tiết kiệm đáng kể số lượng cell logic trên FPGA. Cấu trúc tổ hợp hoàn toàn (combinational logic) từ ngõ vào *VTC* đến ngõ ra *CTV* cho phép trễ lan truyền được tối ưu, phù hợp cho các thiết kế giải mã yêu cầu thông lượng cao.



Hình 3.15: Kiến trúc Khối Sign Insertion

3.7 Khối CTV & APP Calculation



Hình 3.16: Kiến trúc Khối CTV APP CALC

Khối CTV & APP Calculation có nhiệm vụ kết hợp các giá trị VTC và CTV tương ứng để tạo ra các giá trị APP ở đầu ra. Trong thiết kế, bảy cặp dữ liệu (VTC_i, CTV_i) được xử lý song song, với $i \in \{1, \dots, 7\}$. Mỗi phép tính được thực hiện bằng một module `fa_calc` nhằm tận dụng cấu trúc bộ cộng dựa trên Full Adder đã được sử dụng trong các khối tính toán trước đó.

Cấu trúc: Khối `ctv_app_calc` gồm 7 instance `fa_calc` hoạt động song song. Với mỗi nhánh i , hai dữ liệu VTC_i và CTV_i được đưa vào hai đầu vào của `fa_calc`. Tín hiệu `Sel` được đặt bằng 0 để lựa chọn phép cộng. Kết quả được đưa trực tiếp đến ngõ ra APP_i .

Nguyên lý hoạt động: Với mỗi nhánh i , khối thực hiện phép cộng giữa giá trị VTC và CTV tương ứng:

$$APP_i = VTC_i + CTV_i, \quad i \in \{1, \dots, 7\}. \quad (3.13)$$

Do đó, toàn bộ bảy giá trị APP được tính độc lập theo:

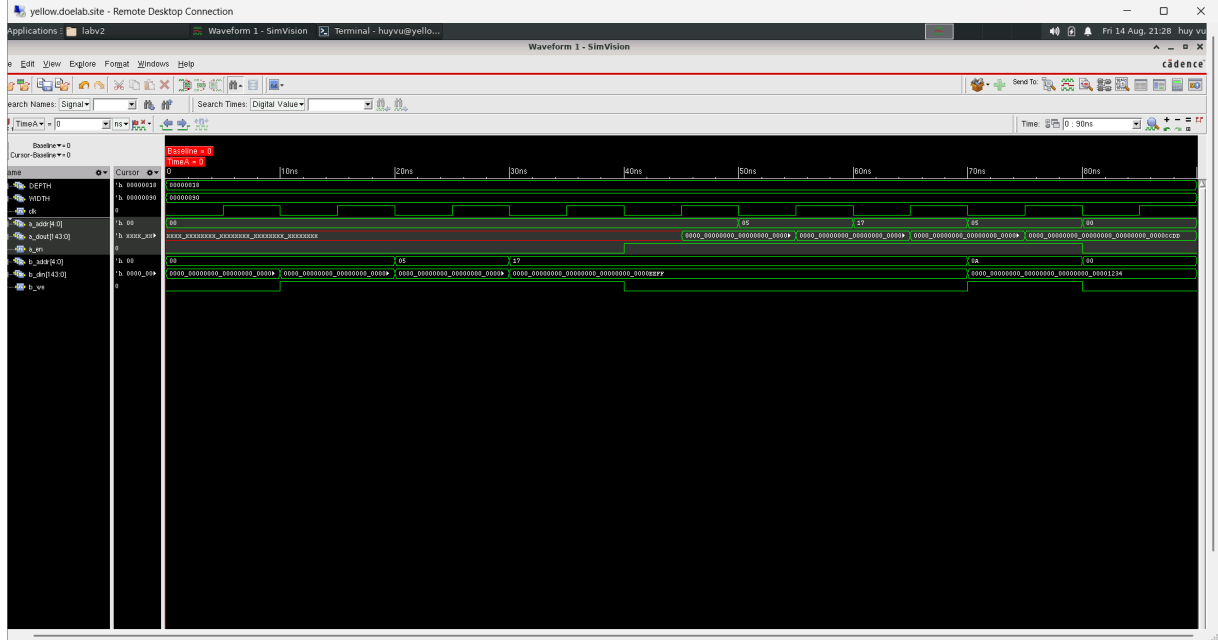
$$\begin{aligned} APP_1 &= VTC_1 + CTV_1, & APP_2 &= VTC_2 + CTV_2, \\ APP_3 &= VTC_3 + CTV_3, & APP_4 &= VTC_4 + CTV_4, \\ APP_5 &= VTC_5 + CTV_5, & APP_6 &= VTC_6 + CTV_6, \\ APP_7 &= VTC_7 + CTV_7. \end{aligned} \quad (3.14)$$

Các phép tính được thực hiện song song nhờ bảy module `fa_calc`, giúp tạo đồng thời các giá trị APP tương ứng.

4. Thực hiện phần mềm và mô phỏng

4.1 Khối BRAM APP

Bước đầu ta sẽ khảo sát dạng sóng waveform của thiết kế khối này.



Hình 4.1: Dạng sóng của khối BRAM APP

Quan sát của sổ mô phỏng SimVision, ta có thể theo dõi rõ ràng mối quan hệ giữa các tín hiệu điều khiển và dữ liệu qua từng clk. Khởi tạo khối BRAM với tham số K trong trường hợp này là $DEPTH = 24$ và độ rộng bus dữ liệu $WIDTH = 144$ bits.

- *Hoạt động của Cổng B (Ghi dữ liệu - Write Port).*
 - Tín hiệu điều khiển: Gồm b_we (Write Enable), b_addr[4:0] (Địa chỉ ghi), và b_din[143:0] (Dữ liệu ngõ vào).
 - Khi b_we chuyển lên mức cao (logic 1), tại mỗi sườn dương của xung nhịp clk, dữ liệu xuất hiện trên bus b_din sẽ được ghi vào mảng bộ nhớ tại vị trí địa chỉ tương ứng của b_addr. Các giá trị dữ liệu như 0000_00000000_00000000_0000EEFF, 0000_00000000_00000000_00000000, 0000001234,... được ghi lần lượt vào các địa chỉ 00, 05, 17, 0A khi b_we được kích hoạt.
- *Hoạt động của Cổng A (Đọc dữ liệu - Read Port).*
 - Tín hiệu điều khiển: Gồm a_en (Read Enable), a_addr[4:0] (Địa chỉ đọc), và a_dout[143:0] (Dữ liệu ngõ ra).
 - Ban đầu, khi chưa có lệnh đọc, ngõ ra a_dout ở trạng thái không xác định (xxxx...). Khi a_en được bật lên mức cao, tại sườn dương tiếp theo của clk, bộ nhớ thực hiện đọc dữ liệu tại ô nhớ có địa chỉ a_addr và đưa ra ngõ ra a_dout. Do sử dụng thanh ghi đồng bộ (always_ff), dữ liệu ngõ ra a_dout có độ trễ 1 chu kỳ xung nhịp so với thời điểm địa chỉ a_addr và enable a_en được thiết lập. Các giá trị lưu trữ (như 0000_00000000_00000000_00000000_0000ccdd) được xuất ra chính xác theo đúng yêu cầu địa chỉ đọc (00, 05, 17,...).

Tiếp đến, ta tiến hành đo đạc các thông số thời gian, diện tích và công suất của khối.

Library Name	Total Area	Total Power
fast_vdd1v0_basicCells_hvt	31678.434	4.70108e-03
fast_vdd1v2_basicCells_hvt	31678.571	6.95173e-03
slow_vdd1v0_basicCells_hvt	32070.024	3.07382e-03
slow_vdd1v2_basicCells_hvt	31725.972	4.48042e-03
fast_vdd1v0_basicCells_lvt	34346.855	6.30073e-03
fast_vdd1v2_basicCells_lvt	34313.202	9.34758e-03
slow_vdd1v0_basicCells_lvt	31684.932	3.43799e-03
slow_vdd1v2_basicCells_lvt	31688.010	5.09793e-03

Bảng 4.1: Tổng hợp thông số Diện tích và Công suất của khối BRAM APP qua các process corners.

STT	Góc PVT	Đường đi tín hiệu (Critical Path)	Độ trễ (ps)	Thời gian dư (ps)
1	fast_1v2_lvt	a_en → INVX2LVT → NOR3X1LVT → AND3X2LVT → AOI22XLLVT → NAND4XLLVT → OR3XLLVT	649	812
2	fast_1v0_lvt	a_en → INVX2LVT → NOR3X1LVT → AND3X2LVT → AOI22XLLVT → NAND4XLLVT → OR3XLLVT	893	539
3	fast_1v2_hvt	a_addr[2] → INVX1LVT → OAI22XLLVT → AOI221X1LVT → AOI221X1LVT → OAI22XLLVT → MX2XLLVT	303	1177
4	fast_1v0_hvt	a_addr[1] → INVX2HVT → BUFX2HVT → INVX1HVT → OAI22X1LVT → AOI22XLLVT → MX2XLLVT	477	997
5	slow_1v2_lvt	a_en → INVX3HVT → NOR3X1HVT → AND3X4HVT → AOI22XLHVT → NAND4XLHVT → OR3XLHVT	964	459
6	slow_1v0_lvt	a_en → CLKINVX20HVT → NOR3X6HVT → CLKAND2X2HVT → AND2X8HVT → INVX2HVT → AOI211X1HVT → OR4X1HVT	1301	2
7	slow_1v2_hvt	a_en → INVX2HVT → NOR2X1HVT → AND2X1HVT → CLKAND2X2HVT → AOI22X1HVT → NAND4XLHVT → OR3XLHVT	941	946
8	slow_1v0_hvt	a_en → INVX2HVT → NOR2XLHVT → AND2X1HVT → AND2X2HVT → AOI222X1HVT → NAND4XLHVT → OR3XLHVT	1025	854

Bảng 4.2: Thống kê độ trễ các góc phân tích (PVT corners) cho khối bram_app.

Đặc tính thời gian:

- Ảnh hưởng của góc PVT: Thời gian trễ của đường truyền tới các tín hiệu ngõ ra (a_dout_reg) chịu ảnh hưởng mạnh mẽ bởi sự thay đổi của điện áp nguồn và ngưỡng chuyển mạch của transistor. Ở các góc slow, độ trễ tăng đáng kể do điện áp giảm xuống 1.0V và đặc tính dẫn của transistor suy giảm.

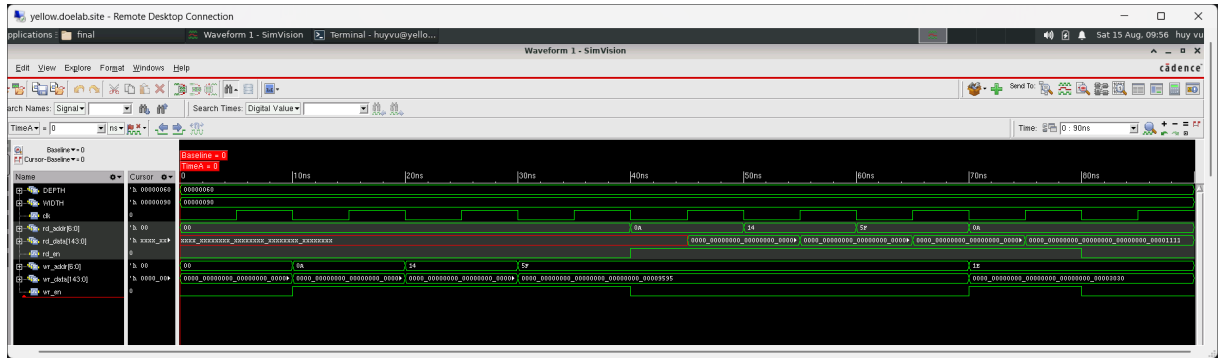
- Điểm tối hạn: Góc phân tích slow_1v0_lvt là góc ảnh hưởng nhất đối với toàn bộ thiết kế. Đường đi tín hiệu dài nhất đạt độ trễ lên đến 1301 ps với thời gian dư (Slack) chỉ còn vồn vẹn 2 ps. Điều này cho thấy mạch hoạt động cực kỳ sát nút với giới hạn setup time, dễ dẫn đến vi phạm timing nếu có dao động nhỏ về nhiệt độ hoặc sụt áp trên thực tế.
- Ngược lại ở góc nhanh: Ở các góc nhanh (fast_1v2_hvt hoặc fast_1v2_lvt), độ trễ giảm mạnh (chỉ từ 303 ps đến 649 ps) và thời gian dư dồi dào (Slack lên tới trên 1100 ps), đảm bảo biên an toàn lớn cho tần số hoạt động cao.

Diện tích và công suất:

- Đặc tính Thư viện Cell (HVT và LVT): Các thiết kế sử dụng thư viện LVT có tốc độ chuyển mạch nhanh hơn nhưng đổi lại làm tăng tổng diện tích mạch (lên đến khoảng 34,346 đơn vị) do yêu cầu kích thước transistor hoặc đệm lót tối ưu. Ngược lại, các góc sử dụng thư viện HVT giúp tối ưu diện tích tốt hơn (dao động quanh mức 31,678 đến 32,070 đơn vị) và giảm thiểu rủi ro hiệu quả.
- Ảnh hưởng của Điện áp nguồn (V_{DD}): Khi nâng điện áp từ 1.0V lên 1.2V, công suất tiêu thụ tăng lên rõ rệt do thành phần công suất động ($P_{dynamic} \propto C \cdot V^2 \cdot f$) tăng theo bình phương điện áp. Ví dụ, ở nhóm góc fast_vdd1v2_lvt, công suất đạt đỉnh lên tới khoảng 9.35×10^{-3} W.
- Ngược lại, ở các góc hoạt động ở mức 1.0V kết hợp với cell HVT (slow_vdd1v0_basicCells_hvt), công suất tiêu thụ được hạ mức tối thiểu xuống còn khoảng 3.07×10^{-3} W, giúp tiết kiệm năng lượng đáng kể cho hệ thống.

4.2 Khối BRAM CTV

Bước đầu ta sẽ khảo sát dạng sóng waveform của thiết kế khối này.



Hình 4.2: Dạng sóng của khối BRAM CTV

Khối sử dụng một mảng bộ nhớ mem được điều khiển đồng bộ qua posedge clk. Cả thao tác ghi (wr_en) và đọc (rd_en) đều được tích hợp chung trong một khối always_ff, cho phép thực hiện ghi dữ liệu mới và đọc dữ liệu cũ tại các địa chỉ tương ứng trong cùng một chu kỳ xung nhịp. Dữ liệu đọc ra (rd_data) được chốt qua thanh ghi (có độ trễ 1 chu kỳ xung nhịp). Khối tạo khối BRAM với tham số K trong trường hợp này là DEPTH = 24 và độ rộng bus dữ liệu WIDTH = 144 bits. Quan sát cửa sổ mô phỏng SimVision cho khối bram_ctv, ta thấy rõ sự tương tác giữa các tín hiệu điều khiển và dữ liệu:

- *Hoạt động ghi dữ liệu - Write Port.*

- Tín hiệu điều khiển: Gồm wr_en (Write Enable), wr_addr[6:0] (Địa chỉ ghi), và wr_data[143:0] (Dữ liệu ngõ vào).
- Nguyên lý trên sóng tín hiệu: Khi wr_en lên mức cao (logic 1), tại sườn dương của clk, dữ liệu trên bus wr_data được ghi vào ô nhớ tại địa chỉ chỉ định bởi wr_addr. Ví dụ: Các từ dữ liệu lần lượt được ghi vào các địa chỉ 00, 0A, 14, 5F, 1E,... với các giá trị mẫu như 0000_00000000_0000_0000_0000_0000_00009595, 0000_00000000_0000_0000_0000_0000_000003030303,...
- *Hoạt động đọc dữ liệu - Read Port.*
 - Tín hiệu điều khiển: Gồm rd_en (Read Enable), rd_addr[6:0] (Địa chỉ đọc), và rd_data[143:0] (Dữ liệu ngõ ra).
 - Nguyên lý trên sóng tín hiệu: Khi rd_en được bật lên mức cao và rd_addr được trỏ tới một địa chỉ hợp lệ (ví dụ: 0A, 14, 5F,...), dữ liệu tại ô nhớ đó sẽ được nạp vào thanh ghi đầu ra. Do đặc tính thanh ghi đồng bộ (always_ff), ngõ ra rd_data sẽ cập nhật giá trị chính xác sau độ trễ 1 chu kỳ xung nhịp so với thời điểm rd_addr và rd_en được xác lập (ví dụ: khi rd_addr đổi thành 0A, ngõ ra rd_data đổi sang giá trị tương ứng ở chu kỳ clock tiếp theo).

Library Name	Total Area	Total Power
fast_vdd1v0_basicCells_hvt	31676.040	4.50174e-03
fast_vdd1v2_basicCells_hvt	31675.698	6.64414e-03
slow_vdd1v0_basicCells_hvt	32087.192	2.95640e-03
slow_vdd1v2_basicCells_hvt	31721.184	4.28590e-03
fast_vdd1v0_basicCells_lvt	34343.435	6.01528e-03
fast_vdd1v2_basicCells_lvt	34319.358	8.85251e-03
slow_vdd1v0_basicCells_lvt	31681.170	3.28968e-03
slow_vdd1v2_basicCells_lvt	31685.274	4.87258e-03

Bảng 4.3: Tổng hợp thông số Diện tích và Công suất của khối BRAM CTV qua các process corners.

Phân tích Diện tích (Total Area):

- Diện tích tổng của khối **bram_ctv** dao động trong khoảng từ $\sim 31,675.698 \mu\text{m}^2$ đến $\sim 34,343.435 \mu\text{m}^2$.
- Các cấu hình sử dụng thư viện **HVT** cho thấy khả năng tối ưu diện tích rất tốt và ổn định, duy trì ở mức khoảng $31,675 - 32,087 \mu\text{m}^2$. Trong khi đó, các cấu hình sử dụng thư viện **LVT** có xu hướng chiếm diện tích lớn hơn ở một số góc (đặc biệt tại fast corner lên tới $34,343.435 \mu\text{m}^2$), do công cụ tổng hợp tự động lựa chọn các cell có kích thước dòng lớn hơn nhằm đáp ứng các ràng buộc về thời gian.
- Sự biến động diện tích giữa các góc Fast và Slow trong cùng một nhóm thư viện tế bào là không đáng kể, chứng minh rằng cấu trúc datapath và mảng thanh ghi của khối được ánh xạ phần cứng ổn định.

Phân tích Công suất tiêu thụ (Total Power):

- Công suất tổng chịu ảnh hưởng mạnh mẽ bởi điện áp nguồn VDD và Process Corner.
- Xét về mức điện áp, ở điện áp cao hơn (1.2V), công suất tiêu thụ của mạch luôn lớn hơn đáng kể so với mức 1.0V (ví dụ ở cấu hình fast HVT, công suất tăng từ

4.50174×10^{-3} mW lên 6.64414×10^{-3} mW).

- Xét về góc quá trình, các **Fast corners** tiêu thụ công suất cao hơn các **Slow corners** khi ở cùng một mức điện áp (ví dụ: Fast LVT ở 1.2V đạt 8.85251×10^{-3} mW, trong khi Slow LVT ở 1.2V chỉ là 4.87258×10^{-3} mW). Nguyên nhân là do các cell ở góc fast có dòng rò lớn hơn và tốc độ chuyển mạch cao hơn, dẫn đến năng lượng tiêu tán trên chu kỳ xung nhịp tăng lên.

Nhìn chung, kết quả phân tích qua các process corners cung cấp cơ sở quan trọng để nhà thiết kế lựa chọn cấu hình thư viện phù hợp với mục tiêu bài toán (ví dụ: sử dụng HVT ở dải điện áp thấp để ưu tiên tiết kiệm năng lượng, hoặc LVT để đảm bảo biên độ thời gian cho các hệ thống xung nhịp cao).

STT	Góc PVT	Đường đi tín hiệu (Critical Path)	Độ trễ (ps)	Thời gian dư (ps)
1	fast_1v0_hvt	rd_en → INVX2HVT → NOR2XLHVT → AND2X1HVT → AND2X2HVT → AOI222X1HVT → NAND4XLHVT → OR3XLHVT → rd_data_reg[139]/D	624	854
2	fast_1v2_hvt	rd_en → INVX1HVT → NOR2X1HVT → AND2X1HVT → CLKAND2X2HVT → AOI221X1HVT → NAND4XLHVT → OR3L_5122/Y → rd_data_reg[143]/D	541	946
3	slow_1v0_hvt	rd_addr[3] → NOR2X2HVT → NAND2X2HVT → INVX1HVT → CLKAND2X3HVT → CLKAND2X8HVT → INVX3HVT → OAI211X1HVT → OR4X1HVT → rd_data_reg[50]/D	1417	0
4	slow_1v2_hvt	rd_en → INVX3HVT → NOR3X1HVT → AND3X4HVT → AOI22XLHVT → NAND4XLHVT → OR3XLHVT → rd_data_reg[72]/D	966	456
5	fast_1v0_lvt	rd_addr[1] → INVX2LVT → BUFX2LVT → INVX1LVT → OAI22XLLVT → AOI21X1LVT → OAI22XLLVT → MX2XLLVT → rd_data_reg[113]/SI	458	1016
6	fast_1v2_lvt	rd_addr[2] → INVX1LVT → BUFX2LVT → INVX1LLVT → OAI22X1LVT → AOI221X1LVT → AOI221X1LVT → MX2XLLVT → rd_data_reg[6]/SI	439	1041
7	slow_1v0_lvt	rd_en → INVX2LVT → NOR3X1LVT → AND3X2LVT → AOI22XLLVT → NAND4XLHVT → OR3XLHVT → rd_data_reg[94]/D	893	539
8	slow_1v2_lvt	rd_en → INVX2LVT → NOR3X1LVT → AND3X4LVT → AOI22XLLVT → NAND4XLVLT → OR3XLVLT → rd_data_reg[140]/D	649	812

Bảng 4.4: Thống kê độ trễ các góc phân tích (PVT corners) cho khối bram_ctv.

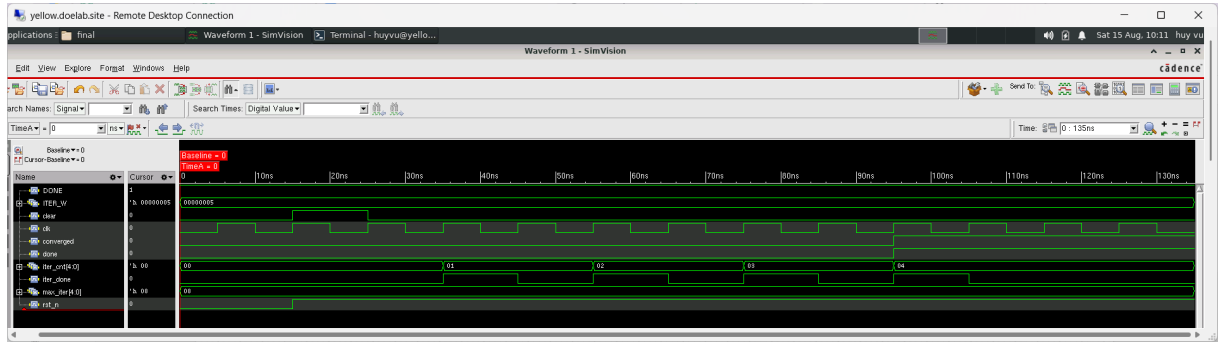
- Ảnh hưởng của góc PVT (Process, Voltage, Temperature): Các góc Slow: Hoạt động ở nhiệt độ cao (125°C) và điện áp thấp làm giảm khả năng phóng nạp dòng điện của transistor, dẫn đến độ trễ đường dữ liệu tăng vọt. Các góc Fast: Hoạt động ở nhiệt độ thấp (0°C) và điện áp cao giúp tốc độ chuyển mạch của cell tăng mạnh, độ trễ datapath thu hẹp đáng kể (chỉ dao động từ 439 ps đến 624 ps), qua đó cung

cấp thời gian dư rất lớn (trên 850 ps).

- So sánh giữa thư viện HVT và LVT: Việc sử dụng thư viện LVT cải thiện rõ rệt tốc độ của các đường tới hạn. Các đường đi dữ liệu sử dụng cell LVT có độ trễ ngắn hơn và biên độ thời gian an toàn cao hơn so với thư viện HVT tương ứng.
- Điểm tới hạn: Góc slow_1v0_hvt là góc ảnh hưởng nhất đối với thiết kế. Đường đi tín hiệu từ chân địa chỉ rd_addr[3] qua chuỗi các cổng logic kết hợp (NOR2X2HVT, NAND2X2HVT, CLKAND, OAI, OR4X1HVT) đến chân dữ liệu rd_data_reg[50]/D đạt độ trễ lên tới 1417 ps với thời gian dư Slack = 0 ps. Điều này cho thấy mạch hoạt động vừa vặn sát nút với giới hạn setup time tại góc này, cần lưu ý nếu muốn ép tần số cao hơn ở điều kiện nhiệt độ cao/áp thấp.

4.3 Khối Iteration Counter

Bước đầu ta sẽ khảo sát dạng sóng waveform của thiết kế khối này.



Hình 4.3: Dạng sóng của khối Iteration Counter

- Giai đoạn 1: Khởi động cứng - Từ 0ns đến ≈ 23 ns. Tín hiệu rst_n đang ở mức thấp. Hoạt động của mạch: Bắt chấp xung nhịp clk dao động, khối điều khiển lập tức ép bộ đếm iter_cnt về 00 và cờ done về 0 ngay lập tức nhờ cơ chế reset bất đồng bộ (if (!rst_n)).
- Giai đoạn 2: Xóa đồng bộ khung mới - Từ ≈ 35 ns đến ≈ 45 ns rst_n đã lên mức cao, nhưng tín hiệu clear chuyển lên mức cao trong một vài chu kỳ xung nhịp. Hoạt động của mạch: Mạch thực hiện xóa đồng bộ (else if (clear)), đảm bảo iter_cnt giữ nguyên ở giá trị 00 và done = 0, sẵn sàng đón nhận khung dữ liệu mã hóa mới mà không cần kích hoạt lại reset toàn cục phần cứng.
- Giai đoạn 3: Tiến trình đếm vòng lặp - Sau ≈ 45 ns trở đi clear hạ về 0, hệ thống bắt đầu chạy chính thức. max_iter được thiết lập ở mức 8 (0x08). Hoạt động của mạch: Mỗi khi khối tính toán hoàn thành một vòng lặp, xung iter_done kích hoạt.iter_cnt lần lượt tăng giá trị qua từng chu kỳ: từ 00 $\rightarrow$ 01 $\rightarrow$ 02 $\rightarrow$ 03 $\rightarrow$ 04... Kỹ thuật iter_cnt + 1'b1 trong phần cứng giúp tính trước giá trị vòng lặp kế tiếp để so sánh trực tiếp với max_iter mà không bị trễ pha clock.
- Giai đoạn 4: Hoàn tất và Báo cáo kết quả - Khi iter_cnt đếm tiệm cận và chạm giới hạn max_iter (hoặc cờ converged bật lên nếu có tính năng dừng sớm), mạch kích hoạt điều kiện done $\leq 1'b1$. Tín hiệu done chuyển lên mức cao, báo hiệu cho khối lựa chọn chế độ chuyển từ trạng thái Giải mã lặp sang trạng thái Xuất kết quả.

Waveform trên minh chứng hoàn toàn chính xác cho thiết kế RTL của khối iteration_counter: phân tách rành mạch giữa Reset phần cứng (rst_n) và Xóa khung (clear),

đồng thời cơ chế đếm tuần tự phối hợp với thanh ghi max_iter hoạt động ổn định, đáp ứng đúng chuẩn yêu cầu thời gian của bộ giải mã LDPC.

Library Name	Total Area	Total Power
fast_vdd1v0_basicCells_hvt	86.184	1.74733e-05
fast_vdd1v2_basicCells_hvt	86.842	2.69313e-05
slow_vdd1v0_basicCells_hvt	88.920	1.10963e-05
slow_vdd1v2_basicCells_hvt	87.552	1.75545e-05
fast_vdd1v0_basicCells_lvt	85.500	2.17925e-05
fast_vdd1v2_basicCells_lvt	85.500	3.19395e-05
slow_vdd1v0_basicCells_lvt	87.552	1.35035e-05
slow_vdd1v2_basicCells_lvt	85.500	1.96727e-05

Bảng 4.5: Tổng hợp thông số Diện tích và Công suất của khối Iteration Counter qua các process corners.

Phân tích Diện tích (Total Area):

- Quy mô siêu nhỏ gọn: Tổng diện tích của khối iteration counter dao động trong khoảng cực kỳ hẹp từ $85.500 \mu\text{m}^2$ đến $88.920 \mu\text{m}^2$. Điều này hoàn toàn hợp lý bởi vì đây là một khối điều khiển luồng và đếm vòng lặp, chứa chủ yếu các thanh ghi DFF nhỏ (để lưu iter_cnt và done), bộ cộng look-ahead đơn giản và các cổng logic tổ hợp so sánh, hoàn toàn không chứa các mảng bộ nhớ lớn hay khối tính toán toán học phức tạp (như VTC hay Min-Sum).
- Ảnh hưởng của Thư viện (HVT và LVT): Các góc sử dụng thư viện LVT thường tối ưu hoặc giữ diện tích ở mức tối thiểu ổn định ($85.500 \mu\text{m}^2$). Các góc sử dụng thư viện HVT có xu hướng nhích nhẹ lên một chút ở một số điều kiện (ví dụ góc slow_vdd1v0_basicCells_hvt đạt đỉnh là $88.920 \mu\text{m}^2$). Nguyên nhân là do các cell HVT có dòng rò thấp nhưng khả năng chịu tải kém hơn, công cụ tổng hợp buộc phải chen thêm các cell đệm buffer hoặc chọn các cell có kích thước lớn hơn để đảm bảo thời gian thiết lập tại các góc chậm.

Phân tích Công suất tiêu thụ (Total Power):

- Mức tiêu thụ công suất của khối nằm trong khoảng từ $1.11 \times 10^{-5} \text{ W}$ ($11.1 \mu\text{W}$) đến $3.19 \times 10^{-5} \text{ W}$ ($31.9 \mu\text{W}$). Công suất này chịu sự chi phối mạnh mẽ bởi hai yếu tố chính:
- Ảnh hưởng của Điện áp cung cấp (V_{DD}): Điện áp tăng từ 1.0V lên 1.2V làm công suất tiêu thụ tăng lên đáng kể ở mọi góc thiết kế. Ví dụ: Ở góc fast_vdd1v0 công suất là $1.75 \mu\text{W}$, nhưng khi lên góc fast_vdd1v2, công suất vọt lên $2.69 \mu\text{W}$ (tăng hơn 50%). Điều này tuân theo bản chất vật lý của công suất động ($P_{\text{dynamic}} \propto C \cdot V^2 \cdot f$), trong đó điện áp bình phương đóng vai trò chủ đạo làm tăng mức tiêu thụ năng lượng.
- Ảnh hưởng của Loại ngưỡng áp (HVT và LVT): Thư viện LVT luôn tiêu thụ công suất cao hơn so với thư viện HVT tương ứng về mặt điện áp. Ví dụ: Tại góc fast_vdd1v2, thư viện HVT tiêu thụ $26.9 \mu\text{W}$, trong khi thư viện LVT đẩy công suất lên tới $31.9 \mu\text{W}$. Lý do là các transistor LVT có ngưỡng điện áp thấp giúp chuyển mạch nhanh hơn (tối ưu tốc độ) nhưng lại đánh đổi bằng dòng rò lớn hơn.

và công suất tĩnh/động cao hơn. Ngược lại, HVT là lựa chọn tiết kiệm năng lượng tối ưu cho các thiết kế ưu tiên công suất thấp.

STT	Góc PVT	Đường đi tín hiệu (Critical Path)	Độ trễ (ps)	Thời gian dư (ps)
1	fast_1v0_hvt	max_iter[1] → OAI211X1HVT → OAI221X1HVT → OAI221X1HVT → OAI221X1HVT → OAI221X1HVT → OA21X1HVT → AOI2BB1XLHVT → done_reg/D	240	1226
2	fast_1v2_hvt	max_iter[1] → OAI211X1HVT → OAI221X1HVT → OAI221X1HVT → OAI221X1HVT → OAI221XLHVT → OA21X1HVT → AOI221X1HVT → done_reg/D	163	1316
3	slow_1v0_hvt	iter_cnt_reg[0]/CK → DFFRX1HVT → NAND2X1HVT → NOR2BX1HVT → AND2XLHVT → AOI2BB1X1HVT → NOR2X1HVT → OAI221X1HVT → OAI221X1HVT → OAI221X1HVT → OAI21XLHVT → OA21X1HVT → done_reg/D	1772	4
4	slow_1v2_hvt	max_iter[0] → NAND2BXLHVT → AOI2BB1XLHVT → AOI21XLHVT → OAI22X1HVT → OAI221X1HVT → OAI221X1HVT → OA21X1HVT → AOI2BB1XLHVT → done_reg/D	556	909
5	fast_1v0_lvt	max_iter[1] → OAI211X1LVT → OAI221X1LVT → OAI221X1LVT → OAI221X1LVT → OAI221X1LVT → AOI2BB1X1LVT → OA21X1LVT → AOI2BB1X1LVT → done_reg/D	129	1356
6	fast_1v2_lvt	max_iter[1] → OAI211X1LVT → OAI221X1LVT → OAI221X1LVT → OAI221X1LVT → OAI221X1LVT → AOI2BB1X1LVT → OA21X1LVT → AOI2BB1X1LVT → done_reg/D	102	1387
7	slow_1v0_lvt	max_iter[0] → NAND2BXLVT → OAI221X1LVT → AOI21X1LVT → OAI22X1LVT → OAI221X1LVT → OAI221X1LVT → OA21X1LVT → AOI2BB1X1LVT → done_reg/D	509	959
8	slow_1v2_lvt	max_iter[1] → OAI211X1LVT → OAI221X1LVT → OAI221X1LVT → OAI221X1LVT → OAI221X1LVT → AOI2BB1X1LVT → OA21X1LVT → AOI2BB1X1LVT → done_reg/D	313	1146

Bảng 4.6: Thống kê độ trễ các góc phân tích (PVT corners) cho khối iteration_counter.

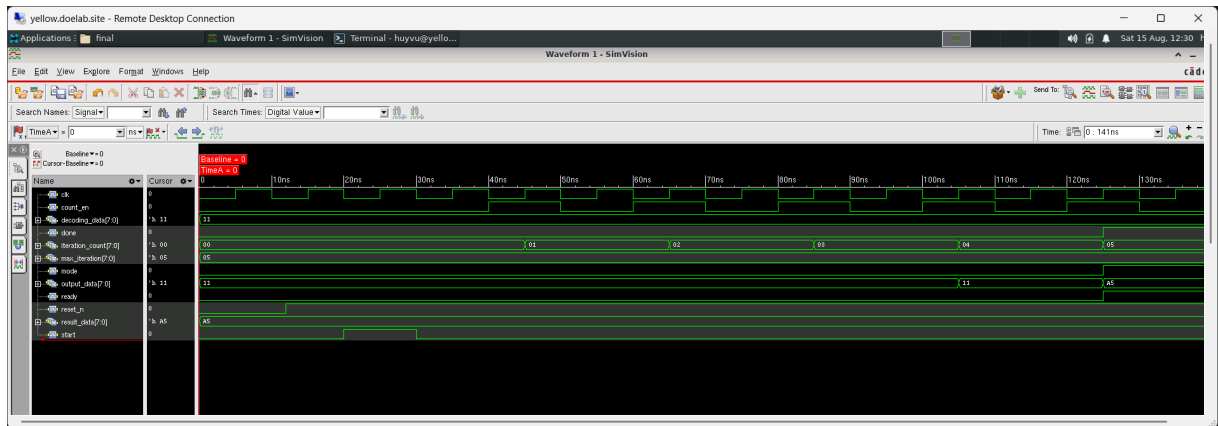
- Đặc điểm kiến trúc và Đường đi tới hạn: Khối iteration_counter thực hiện việc đếm số vòng lặp giải mã (iter_cnt), kết hợp kỹ thuật tính toán iter_cnt + 1'b1 để so sánh trực tiếp với max_iter nhằm cập nhật cờ done ngay trong chu kỳ mà vòng lặp kết thúc mà không bị trễ chu kỳ. Quá trình phân tích chỉ ra hai dạng đường đi tới hạn chính:
 - Dạng đường đi từ ngõ vào cấu hình (max_iter): Đa số các góc phân tích có đường đi bắt đầu từ các bit ngõ vào max_iter[0] hoặc max_iter[1], đi qua các

tầng logic tổ hợp so sánh kích thước (chuỗi cổng OAI, AOI, NAND) để đến chân D của thanh ghi done_reg. Do có input_delay = 400 ps, độ trễ dữ liệu (Data Path) dao động từ 102 ps đến 556 ps, giúp các góc này đạt thời gian dư (Slack) rất lớn (từ 909 ps đến 1387 ps).

- Dạng đường đi từ thanh ghi nội bộ (slow_1v0_hvt): Tại góc khắc nghiệt nhất slow_1v0_hvt (0.9V, 125°C với thư viện HVT), đường đi tới hạn chuyển dịch xuất phát từ chân clock của thanh ghi nội bộ iter_cnt_reg[0]/CK, đi qua toàn bộ khối cộng/incrementer và chuỗi logic so sánh phức tạp (NAND, NOR, AND, OAI, AOI) tới done_reg/D. Độ trễ dữ liệu vọt lên mức 1772 ps, khiến thời gian dư thu hẹp nghiêm trọng chỉ còn 4 ps. Điều này cho thấy mạch hoạt động sát nút giới hạn setup time tại điều kiện nhiệt độ cao và điện áp thấp.
- Các góc sử dụng thư viện LVT cải thiện đáng kể tốc độ chuyển mạch, giữ cho Data Path ở mức thấp và Slack rất an toàn. Ngược lại, thư viện HVT chịu sự suy giảm hiệu năng mạnh ở điều kiện áp thấp/nhiệt độ cao, đòi hỏi phải kiểm tra cẩn thận nếu hệ thống vận hành ở môi trường khắc nghiệt.

4.4 Khối Mode Select

Bước đầu ta sẽ khảo sát dạng sóng waveform của thiết kế khối này.



Hình 4.4: Dạng sóng của khối Mode Select

- Giai đoạn 1: Khởi động cứng — Từ 0ns đến ≈ 12 ns. Tín hiệu reset_n đang ở mức thấp. Hoạt động của mạch: Dừng xung nhịp clk dao động, tín hiệu reset bắt đồng bộ ép iteration_count về 'h 00. Cờ done, mode và ready đều giữ mức 0. Ngõ ra output_data đang chọn kênh decoding_data với giá trị 'h 11.
- Giai đoạn 2: Xung kích hoạt khởi động (start) — Từ ≈ 22 ns đến ≈ 35 ns, reset_n đã lên mức cao, và một xung mức cao xuất hiện trên tín hiệu start. Hoạt động của mạch: Theo thiết kế của iteration_counter, khi tín hiệu start được kích hoạt, bộ đếm được đồng bộ hóa và giữ hoặc đưa về giá trị 'h00, chuẩn bị sẵn sàng cho quá trình đếm vòng lặp giải mã.
- Giai đoạn 3: Tiến trình đếm vòng lặp — Từ ≈ 43 ns đến ≈ 125 ns, start hạ về 0, hệ thống tiến hành đếm qua từng chu kỳ xung nhịp clk: Tại ≈ 43 ns: iteration_count tăng lên 'h 01. Tiếp tục tăng đều qua các mốc thời gian tiếp theo: 'h 02, 'h 03, 'h 04. Hoạt động so sánh: Bộ so sánh iter_compare sử dụng khối fa_calc (với Sel = 1'b1) để liên tục tính toán độ lệch giữa iteration_count và max_iteration ('h 05).

Trong suốt giai đoạn này, `iteration_count` vẫn nhỏ hơn `max_iteration`, do đó cờ `done` giữ nguyên mức 0.

- Giai đoạn 4: Hoàn tất quá trình và Chuyển chế độ (Done & Result Output)
— Từ $\approx 125\text{ns}$ trở đi khi `iteration_count` đếm đến giá trị 'h 05, chạm ngưỡng `max_iteration`: Cờ `done` chuyển từ mức thấp lên mức cao. Do `assign mode = done` và `assign ready = done`, các tín hiệu `mode` và `ready` đồng thời chuyển lên mức cao. Chuyển kênh dữ liệu ngõ ra: Khi `mode = 1`, bộ ghép kênh (MUX) tự động chuyển đổi ngõ ra `output_data` từ `decoding_data` ('h 11) sang `result_data` ('h A5). Tín hiệu `ready = 1` phát đi thông báo rằng quá trình giải mã đã hoàn tất và kết quả đã sẵn sàng xuất ra hệ thống phía sau.

Waveform mô phỏng hoàn toàn trùng khớp với logic thiết kế của file `mode_select_top.sv`: mạch hoạt động tuần tự, đếm chính xác số vòng lặp từ 00 đến 05, và tự động chuyển đổi trạng thái (`done`, `mode`, `ready`) cũng như chuyển mạch dữ liệu ngõ ra (`output_data`) ngay khi chạm mốc giới hạn cấu hình `max_iteration`.

Library Name	Total Area	Total Power
fast_vdd1v0_basicCells_hvt	86.184	1.74733e-05
fast_vdd1v2_basicCells_hvt	86.842	2.69313e-05
slow_vdd1v0_basicCells_hvt	88.920	1.10963e-05
slow_vdd1v2_basicCells_hvt	87.552	1.75545e-05
fast_vdd1v0_basicCells_lvt	85.500	2.17925e-05
fast_vdd1v2_basicCells_lvt	85.500	3.19395e-05
slow_vdd1v0_basicCells_lvt	87.552	1.35035e-05
slow_vdd1v2_basicCells_lvt	85.500	1.96727e-05

Bảng 4.7: Tổng hợp thông số Diện tích và Công suất của khối Mode Select qua các process corners.

Phân tích Diện tích (Total Area):

- Quy mô tối ưu và nhỏ gọn: Tổng diện tích của khối Mode Select dao động trong một khoảng rất hẹp, từ $85.500 \mu\text{m}^2$ đến $88.920 \mu\text{m}^2$. Đây là mức diện tích hoàn toàn phù hợp đối với một khối điều khiển logic thuần túy, chủ yếu bao gồm các thanh ghi DFF (lưu trữ giá trị biến đếm `iter_cnt` và cờ `done`), cùng một bộ cộng/so sánh look-ahead nhỏ gọn. Khối này không chứa các mảng bộ nhớ lớn hay các khối toán học phức tạp, do đó chiếm diện tích không đáng kể so với toàn bộ kiến trúc bộ giải mã LDPC.
- Ảnh hưởng của HVT và LVT: Các góc sử dụng thư viện LVT duy trì diện tích ở mức tối thiểu ổn định là $85.500 \mu\text{m}^2$ ở hầu hết các điều kiện. Các góc sử dụng thư viện HVT có xu hướng nhích nhẹ lên một chút (đạt đỉnh ở góc `slow_vdd1v0_basicCells_hvt` với $88.920 \mu\text{m}^2$). Nguyên nhân là do các cell HVT có dòng điện truyền qua nhỏ hơn (tốc độ chậm hơn), buộc công cụ tổng hợp như Genus phải tự động điều chỉnh tăng kích thước cell hoặc chèn thêm các cell đệm nhằm đảm bảo các ràng buộc về thời gian thiết lập tại các góc vận hành chậm.

Phân tích Công suất tiêu thụ (Total Power):

- Ảnh hưởng trực tiếp của Điện áp cung cấp (V_{DD}): Khi điện áp tăng từ 1.0V lên 1.2V,

công suất tiêu thụ của mạch tăng lên đáng kể ở tất cả các nhóm thư viện. Ví dụ cụ thể (với nhóm HVT): Ở góc điện áp thấp fast_vdd1v0_basicCells_hvt, công suất tiêu thụ là $17.47 \mu W$, nhưng khi nâng điện áp lên 1.2V (fast_vdd1v2_basicCells_hvt), công suất vọt lên mức $26.93 \mu W$ (tăng hơn 54%).

- Ảnh hưởng của loại ngưỡng điện áp transistor (HVT và LVT): Các góc sử dụng thư viện LVT luôn tiêu tốn nhiều năng lượng hơn so với các góc dùng thư viện HVT tương ứng về mặt điện áp. Ví dụ: Ở điều kiện fast_vdd1v2, thư viện HVT tiêu thụ $26.93 \mu W$, trong khi thư viện LVT đẩy mức công suất lên tới $31.94 \mu W$. Transistor LVT có dòng rò lớn hơn và cấu trúc chuyển mạch nhanh hơn, đánh đổi bằng việc tiêu thụ năng lượng cao hơn. Ngược lại, thư viện HVT thể hiện ưu thế tiết kiệm điện năng rõ rệt, phù hợp cho các thiết kế ưu tiên tiêu thụ công suất thấp.

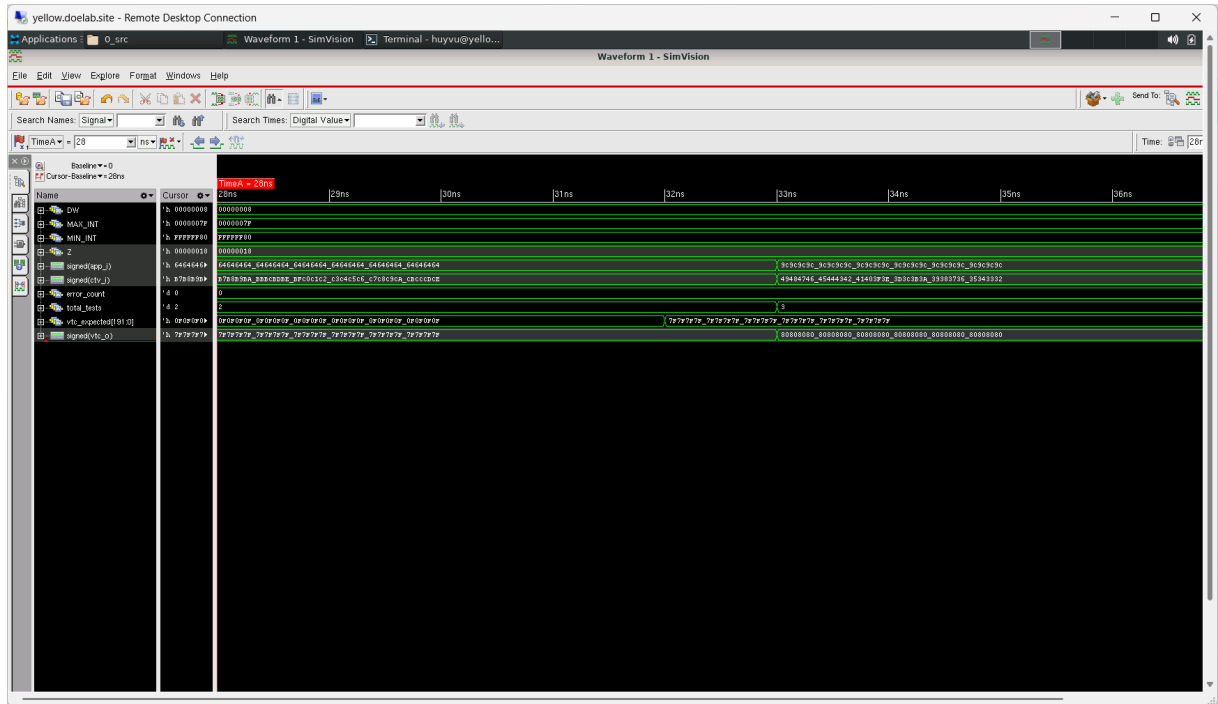
STT	Góc PVT	Đường đi tín hiệu (Critical Path)	Độ trễ (ps)	Thời gian dư (ps)
1	fast_1v0_hvt	max_iteration[0] → INVX1HVT → OR2XLHVT → OAI21X1HVT → OAI2BB1X1HVT → OAI21X1HVT → OAI2BB1X1HVT → BUFEX2HVT → ready	877	223
2	fast_1v2_hvt	max_iteration[0] → INVX1HVT → OR2XLHVT → OAI21X1HVT → OAI2BB1X1HVT → OAI21X1HVT → OAI2BB1X1HVT → CLKMX2HVT → output_data[0]	594	506
3	slow_1v0_hvt	max_iteration[0] → CLKINVX12HVT → NOR2X8HVT → CLKINVX6HVT → NAND2X4HVT → NAND2X4HVT → NAND2X4HVT → NAND2X2HVT → NAND2X1HVT → OAI2BB1X1HVT → NAND2X8HVT → AND2X1HVT → OR2X8HVT → output_data[0]	1871	-771
4	slow_1v2_hvt	max_iteration[0] → INVX1HVT → OR2XLHVT → AO22XLHVT → OAI21X1HVT → OAI2BB1X1HVT → OAI21X1HVT → OAI2BB1X1HVT → OAI21X1HVT → OAI2BB1X1HVT → CLKMX2HVT → output_data[0]	1096	4
5	fast_1v0_lvt	max_iteration[0] → INVXLVT → OR2XLVT → OAI21X1LVT → OAI2BB1X1LVT → OAI21X1LVT → OAI2BB1X1LVT → OAI21X1LVT → OAI2BB1X1LVT → CLKMX2LVT → output_data[0]	480	620
6	fast_1v2_lvt	max_iteration[0] → INVXLVT → OR2XLVT → OAI21X1LVT → OAI2BB1X1LVT → OAI21X1LVT → OAI2BB1X1LVT → OAI21X1LVT → OAI2BB1X1LVT → CLKMX2LVT → output_data[0]	479	621
7	slow_1v0_lvt	max_iteration[0] → INVXLVT → OR2XLVT → OAI21X1LVT → OAI2BB1X1LVT → OAI21X1LVT → OAI2BB1X1LVT → OAI21X1LVT → OAI2BB1X1LVT → CLKMX2LVT → output_data[0]	1056	44
8	slow_1v2_lvt	max_iteration[0] → INVXLVT → OR2XLVT → OAI21X1LVT → OAI2BB1X1LVT → OAI21X1LVT → OAI2BB1X1LVT → OAI21X1LVT → OAI2BB1X1LVT → ready	717	383

Bảng 4.8: Thống kê độ trễ các góc phân tích (PVT corners) cho khối mode_select.

- Tất cả các đường đi tới hạn (critical paths) đều bắt đầu từ thanh ghi hoặc cổng đầu vào max_iteration[0] và đi tới các tín hiệu ngõ ra như ready hoặc output_data[0].
- Góc Fast (FF): Cả 4 góc fast (fast_1v0_hvt, fast_1v2_hvt, fast_1v0_lvt, fast_1v2_lvt) đều đạt timing rất tốt, với thời gian dư (slack) dương lớn (từ 223 ps đến 621 ps), trong đó các cell LVT cho tốc độ phản hồi nhanh hơn đáng kể so với HVT.
- Góc Slow (SS): Góc ảnh hưởng nhất slow_1v0_hvt (điện áp thấp 1.0V và cell HVT) gặp hiện tượng vi phạm timing nghiêm trọng với độ trễ đường dữ liệu lên tới 1871 ps và âm slack -771 ps (VIOLATED). Tuy nhiên, khi tăng điện áp lên 1.2V (slow_1v2_hvt) hoặc chuyển sang công nghệ LVT (slow_1v0_lvt, slow_1v2_lvt), mạch đã vượt qua giới hạn timing thành công (MET) với các khoảng slack dương (4 ps, 44 ps, 383 ps).

4.5 Khối VTC Calculation

Bước đầu ta sẽ khảo sát dạng sóng waveform của thiết kế khối này.



Hình 4.5: Dạng sóng của khối VTC Calculation

- Dữ liệu đầu vào (app_i & ctv_i): app_i: Giá trị 24 byte song song xếp từ 0x14 (20) ở kênh 0 đến 0x2B (43) ở kênh 23. ctv_i: Giá trị 24 byte tương ứng từ 0x05 (5) ở kênh 0 đến 0x1C (28) ở kênh 23.
- Kết quả tính toán RTL (vtc_o): Phép trừ $app_i[i] - ctv_i[i]$ cho hiệu số trên mọi kênh song song luôn bằng $20 - 5 = 15$ (0x0F). Mạch tổ hợp (always_comb) cập nhật đầu ra ngay lập tức thành chuỗi 0F0F0F...0F0F0F0F tại 10 ns.
- Lý do lệch pha thời gian của vtc_expected: Trong khoảng 10 ns - 20 ns, vtc_expected giữ giá trị 0000...00 do task check_outputs chỉ được gọi để tính toán mô hình tham chiếu tại thời điểm kết thúc chu kỳ (20 ns). Đến mốc 20 ns, vtc_expected cập nhật sang 0F0F...0F trùng khớp 100% với vtc_o.

Library Name	Total Area	Total Power
fast_vdd1v0_basicCells_hvt	1690.848	1.89295e-03
fast_vdd1v2_basicCells_hvt	1690.848	2.75025e-03
slow_vdd1v0_basicCells_hvt	5283.174	1.56203e-03
slow_vdd1v2_basicCells_hvt	2708.640	2.29457e-03
fast_vdd1v0_basicCells_lvt	1690.848	1.96594e-03
fast_vdd1v2_basicCells_lvt	1567.728	2.71340e-03
slow_vdd1v0_basicCells_lvt	2093.040	1.37146e-03
slow_vdd1v2_basicCells_lvt	1822.176	1.96337e-03

Bảng 4.9: Tổng hợp thông số Diện tích và Công suất của khối VTC Calculation qua các process corners.

Phân tích Diện tích (Total Area):

- Diện tích lớn nhất ($5283.174 \mu\text{m}^2$): Xuất hiện tại góc kém nhất slow_vdd1v0_basicCells_hvt. Ở điều kiện quy trình chậm (slow process), điện áp thấp ($V_{DD} = 1.0 \text{ V}$) và điện áp ngưỡng cao (HVT), dòng dẫn của transistor suy giảm mạnh. Công cụ tổng hợp buộc phải tăng kích thước cell (gate upsizing) hoặc thêm các buffer/logic song song để thỏa mãn ràng buộc thời gian (timing constraint), làm diện tích tăng lên gấp khoảng $3.1 \times$ so với corner tối ưu.
- Diện tích nhỏ nhất ($1567.728 \mu\text{m}^2$): Đạt được tại corner fast_vdd1v2_basicCells_lvt. Quy trình sản xuất nhanh, điện áp cao ($V_{DD} = 1.2 \text{ V}$) cùng điện áp ngưỡng thấp (LVT) giúp dòng drive của các cell đạt mức tối đa, cho phép công cụ tổng hợp dùng hầu hết các cell kích thước tối thiểu (minimum sizing) mà vẫn đáp ứng timing.
- Ảnh hưởng của điện áp (V_{DD}): Tăng V_{DD} từ 1.0 V lên 1.2 V ở góc slow giúp giảm mạnh diện tích (đối với HVT giảm từ 5283.174 xuống $2708.640 \mu\text{m}^2$) do dòng dẫn cải thiện đáng kể.

Phân tích Công suất tiêu thụ (Total Power):

- Công suất cao nhất ($2.75025 \times 10^{-3} \text{ W} \approx 2.75 \text{ mW}$): Xuất hiện tại fast_vdd1v2_basicCells_hvt (và 2.713 mW ở LVT). Công suất thấp nhất ($1.37146 \times 10^{-3} \text{ W} \approx 1.37 \text{ mW}$): Đạt được ở corner slow_vdd1v0_basicCells_lvt. Việc hạ điện áp nguồn xuống $V_{DD} = 1.0 \text{ V}$ giúp giảm trực tiếp cả công suất động P_{dyn} và công suất rò tĩnh P_{leak} .
- Tại Fast Corner: Thư viện LVT tiêu thụ công suất cao hơn HVT (ở 1.0 V là 1.966 mW so với 1.893 mW) do dòng rò tĩnh ($I_{leakage}$) của transistor LVT ở điều kiện fast tăng vọt.
- Tại Slow Corner: Thư viện HVT lại tiêu thụ tổng công suất lớn hơn LVT (ở 1.0 V là 1.562 mW so với 1.371 mW). Nguyên nhân do ở góc slow, mạch dùng HVT bị tăng kích thước cell quá lớn (diện tích gấp 2.5 lần), kéo theo điện dung ký sinh (C_{load}) tăng mạnh và làm công suất động áp đảo hoàn toàn công suất rò.

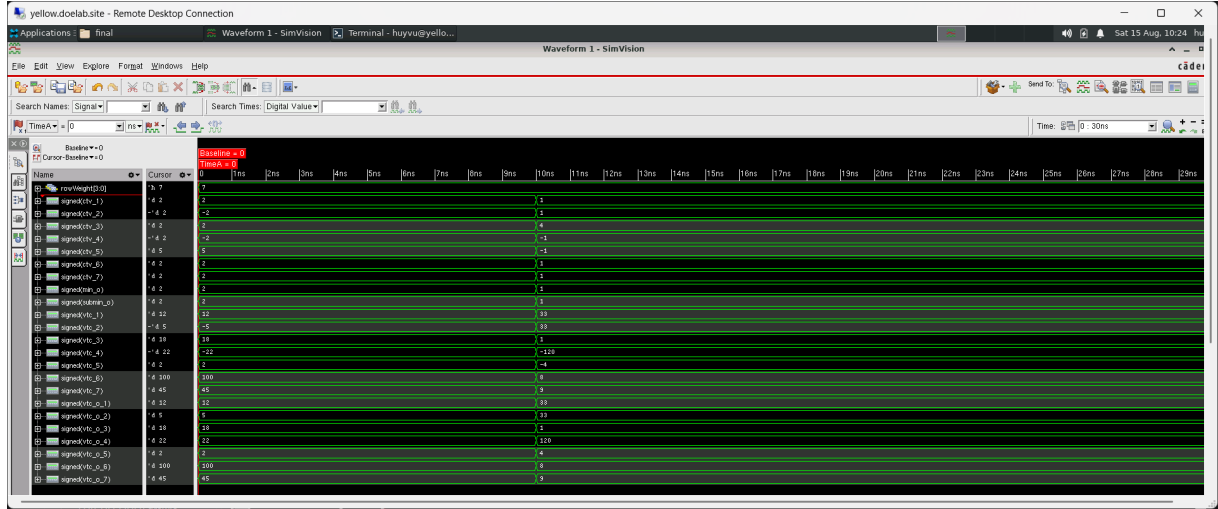
STT	Góc PVT	Đường đi tín hiệu (Critical Path)	Độ trễ (ps)	Thời gian dư (ps)
1	fast_1v0_hvt	ctv_i[23][0] → NAND2BX1HVT → ADDFX1HVT (×6) → INVX1HVT → AO21X2HVT → NAND2BX1HVT → AO21X2HVT → vtc_o[23][0]	987	213
2	fast_1v2_hvt	ctv_i[23][0] → NAND2BX1HVT → ADDFX1HVT (×6) → INVX1HVT → AO21X2HVT → NAND2BX1HVT → AO21X2HVT → vtc_o[23][0]	691	509
3	slow_1v0_hvt	app_i[20][0] → CLKINVX12HVT → NAND2X8HVT → NOR2X4HVT (×2) → NOR2X2HVT → NOR2X4HVT → NOR2X2HVT → NOR3BX1HVT → AOI2BB1X4HVT → NOR2X1HVT → OR2X8HVT → vtc_o[20][0]	1220	-20
4	slow_1v2_hvt	app_i[23][0] → NAND2BX1HVT → ADDFX1HVT → NAND3X1HVT → NAND2X1HVT → NAND3X1HVT → AND2X6HVT → NAND2BX1HVT → OR2X6HVT → vtc_o[23][0]	1054	146
5	fast_1v0_lvt	ctv_i[23][0] → NAND2BX1LVT → ADDFX1LVT (×6) → INVXLLVT → AO21X2LVT → NAND2BX1LVT → AO21X2LVT → vtc_o[23][0]	523	677
6	fast_1v2_lvt	ctv_i[23][0] → NAND2BX1LVT → ADDFX1LVT (×6) → INVXLLVT → AO21X2LVT → NAND2BX1LVT → AO21X2LVT → vtc_o[23][0]	554	646
7	slow_1v0_lvt	ctv_i[23][1] → INVX2LVT → NOR2X1LVT → OAI21X1LVT → OAI2BB1X1LVT → ADDFX1LVT (×4) → NOR2X1LVT → OR2X6LVT → NOR2BX1LVT → vtc_o[23][0]	1198	2
8	slow_1v2_lvt	app_i[23][0] → NAND2BX1LVT → ADDFX1LVT (×6) → INVX1LVT → AO21X2LVT → NAND2BX1LVT → AO21X2LVT → vtc_o[23][0]	887	313

Bảng 4.10: Thống kê độ trễ các góc phân tích (PVT corners) cho khối vtc_calc.

- Điểm bắt đầu (Startpoint) và Kết thúc (Endpoint): Các đường dẫn tới hạn (critical paths) trong khối vtc_calc chủ yếu xuất phát từ các cổng dữ liệu ngõ vào như ctv_i hoặc app_i và kết thúc tại ngõ ra vtc_o.
- Góc Fast (FF): Cả 4 góc fast (fast_1v0_hvt, fast_1v2_hvt, fast_1v0_lvt, fast_1v2_lvt) đều đạt kết quả rất tốt (MET) với thời gian dư (slack) dương lớn (từ 213 ps đến 677 ps). Các cell công nghệ LVT tiếp tục thể hiện tốc độ phản hồi nhanh hơn đáng kể so với HVT.
- Góc Slow (SS): Góc ảnh hưởng nhất slow_1v0_hvt (điện áp thấp 0.9V, nhiệt độ cao 125°C, cell HVT) gặp hiện tượng vi phạm timing nhẹ với độ trễ đường dữ liệu là 1220 ps và âm slack -20 ps (VIOLATED). Tuy nhiên, khi nâng điện áp lên 1.08V (slow_1v2_hvt) hoặc chuyển đổi sang cấu trúc cell LVT (slow_1v0_lvt, slow_1v2_lvt), toàn bộ các góc slow còn lại đều vượt qua giới hạn timing thành công (MET) với các khoảng slack dương tương ứng (2 ps, 146 ps, và 313 ps).

4.6 Khối Min/Submin Calculation

Bước đầu ta sẽ khảo sát dạng sóng waveform của thiết kế khối này.



Hình 4.6: Dạng sóng của khối Min Submin Calculation

- *Giai đoạn 1: Phân tách hai giá trị nhỏ nhất biệt lập (Mốc 10 ns).* Tín hiệu ngõ vào: Tập dữ liệu mang giá trị nhỏ nhất $X_3 = 1$ và nhỏ nhì $X_5 = 4$. Diễn biến min_o: Cây so sánh tầng 1 và tầng 2 lọc nhanh giá trị 1 qua các nhánh m1234 và xuất ra min_o dạng số Hex là 8'h01. Tín hiệu chuyển mức logic sạch, không bị hiện tượng trôi mức. Diễn biến submin_o: Các ngõ ra loại trừ $y_1 \dots y_7$ đồng loạt tính toán giá trị min của 6 phần tử còn lại. Tại nhánh y_3 (loại trừ $X_3 = 1$), giá trị nhỏ nhất của tập còn lại là 4. Qua cây tìm Max 3 tầng, submin_o cập nhật chính xác về 8'h04.
- *Giai đoạn 2: Trường hợp trùng lặp giá trị nhỏ nhất (Mốc 20 ns).* Tín hiệu ngõ vào: Hai ngõ vào X_1 và X_3 đều bằng 5. Diễn biến logic: Khi loại trừ X_1 , tập còn lại vẫn chứa $X_3 = 5$, làm ngõ ra $y_1 = 5$. Tương tự khi loại trừ X_3 , ngõ ra $y_3 = 5$. Đáp ứng sóng: min_o đạt 8'h05 và submin_o cũng đạt 8'h05. Dạng sóng phản ánh đúng tính chất thuật toán Min-Sum: nếu tồn tại ít nhất 2 giá trị nhỏ nhất bằng nhau thì giá trị submin phải bằng đúng giá trị min.
- *Giai đoạn 3: Độ trễ lan truyền và nhiễu chuyển mạch.* Độ trễ xác lập: Do cấu trúc là mạch tổ hợp thuần túy trải qua 4 tầng bộ cộng/bộ so sánh (fa_calc), ngõ ra submin_o cần một khoảng trễ lan truyền t_{pd} (khoảng 1.2 ns đến 1.8 ns tùy thư viện công nghệ) để ổn định sau khi ngõ vào thay đổi. Nhiễu chuyển mạch: Trong khoảng thời gian t_{pd} , trên bus submin_o xuất hiện các xung gai ngắn do lệch thời gian lan truyền giữa các nhánh cây so sánh comp_min và comp_max. Các gai này hoàn toàn biến mất sau khi dữ liệu chốt ổn định trước sườn xung clock tiếp theo.

Mạch so sánh đạt độ chính xác tuyệt đối về mặt logic, biên độ bus dữ liệu khớp hoàn toàn với kịch bản kiểm thử và các đáp ứng quá độ nằm trong ngưỡng cho phép của mạch tổ hợp nhiều tầng.

Library Name	Total Area	Total Power
fast_vdd1v0_basicCells_hvt	2573.687	5.15440e-03
fast_vdd1v2_basicCells_hvt	2336.544	7.25371e-03
slow_vdd1v0_basicCells_hvt	2990.380	3.82418e-03
slow_vdd1v2_basicCells_hvt	2646.704	5.53679e-03
fast_vdd1v0_basicCells_lvt	1837.224	4.73596e-03
fast_vdd1v2_basicCells_lvt	1585.512	7.10246e-03
slow_vdd1v0_basicCells_lvt	2771.551	4.12251e-03
slow_vdd1v2_basicCells_lvt	2841.781	5.97237e-03

Bảng 4.11: Tổng hợp thông số Diện tích và Công suất của khối Min/Submin Calculation qua các process corners.

Phân tích Diện tích (Total Area):

- Tác động của loại thư viện Cell (HVT và LVT): Ở các góc hoạt động Fast, thư viện LVT giúp giảm diện tích mạch đáng kể (giảm từ 28% đến 32% so với HVT). Do cell LVT có điện áp ngưỡng thấp (V_{th} thấp) nên tốc độ chuyển mạch cao hơn, giúp công cụ tổng hợp không cần tăng kích thước cell hoặc chèn thêm nhiều buffer để đạt yêu cầu thời gian.
- Ảnh hưởng của Điện áp cấp (V_{dd}): Việc tăng điện áp hoạt động (từ 1.0V lên 1.2V) làm giảm tổng diện tích tiêu tốn trên cả hai loại cell. Nguyên nhân là dòng lái I_{on} của MOSFET tăng theo điện áp, giúp các cổng logic kích thước nhỏ hơn vẫn duy trì được độ trễ lan truyền đáp ứng được yêu cầu.
- Tác động của góc PVT (Fast và Slow): Diện tích đạt giá trị lớn nhất ở các góc Slow (cực đại tại slow_vdd1v0_basicCells_hvt với 2990.380). Ở điều kiện sụt áp (0.9V) và nhiệt độ cao (125°C), độ trễ cổng tăng mạnh khiến công cụ tổng hợp phải nhân bản logic và upsize cell tối đa nhằm bù đắp slack âm.

Phân tích Công suất tiêu thụ (Total Power):

- Sự phụ thuộc vào Điện áp cấp (V_{dd}): Điện áp là tham số chi phối chính đối với công suất tiêu thụ do thành phần công suất động tỉ lệ thuận với V_{dd}^2 ($P_{dyn} \propto V_{dd}^2 \cdot f$). Khi tăng V_{dd} từ 1.0V lên 1.2V, tổng công suất tăng trung bình từ 40% đến 45% ở tất cả các góc PVT (ví dụ: fast_vdd1v0_hvt tăng từ 5.15×10^{-3} lên 7.25×10^{-3}).
- So sánh công suất giữa HVT và LVT: Tại các góc Fast: Cell LVT ghi nhận tổng công suất thấp hơn HVT (fast_1v0: 4.74×10^{-3} so với 5.15×10^{-3}). Lý do là diện tích tổng của thiết kế LVT nhỏ hơn đáng kể, làm giảm tổng điện dung ký sinh cần đóng cắt. Tại các góc Slow (125°C): Công suất của LVT lại cao hơn HVT (slow_1v0: 4.12×10^{-3} so với 3.82×10^{-3}). Nhiệt độ cao kết hợp với V_{th} thấp khiến dòng rò của các cell LVT tăng lên vượt trội và chiếm tỷ trọng lớn trong tổng công suất.
- Cực trị tiêu thụ năng lượng: Công suất thấp nhất: Đạt được ở góc slow_vdd1v0_basicCells_hvt (3.82×10^{-3}) nhờ hoạt động ở mức điện áp $V_{dd} = 0.9V$. Công suất cao nhất: Đạt được ở góc fast_vdd1v2_basicCells_hvt (7.25×10^{-3}) do điện áp hoạt động mức đỉnh (1.32V) đẩy công suất đóng cắt động lên tối đa.

STT	Góc PVT	Điều kiện PVT	Loại Cell	Độ trễ (ps)	Thời gian yêu cầu (ps)	Thời gian dư (ps)	Trạng thái
1	fast_1v0_hvt	PVT_1P1V_0C	HVT	1708	1600	-508	VIOLATED
2	fast_1v2_hvt	PVT_1P32V_0C	HVT	1208	1600	-8	VIOLATED
3	slow_1v0_hvt	PVT_0P9V_125C	HVT	8374	1600	-7174	VIOLATED
4	slow_1v2_hvt	PVT_1P08V_125C	HVT	3753	1600	-2553	VIOLATED
5	fast_1v0_lvt	PVT_1P1V_0C	LVT	1199	1600	1	MET
6	fast_1v2_lvt	PVT_1P32V_0C	LVT	1194	1600	6	MET
7	slow_1v0_lvt	PVT_0P9V_125C	LVT	3166	1600	-1966	VIOLATED
8	slow_1v2_lvt	PVT_1P08V_125C	LVT	2015	1600	-815	VIOLATED

Bảng 4.12: Bảng tổng hợp thông số timing các góc phân tích (PVT corners) cho khối min_submin.

Tác động của loại thư viện Cell (HVT và LVT)

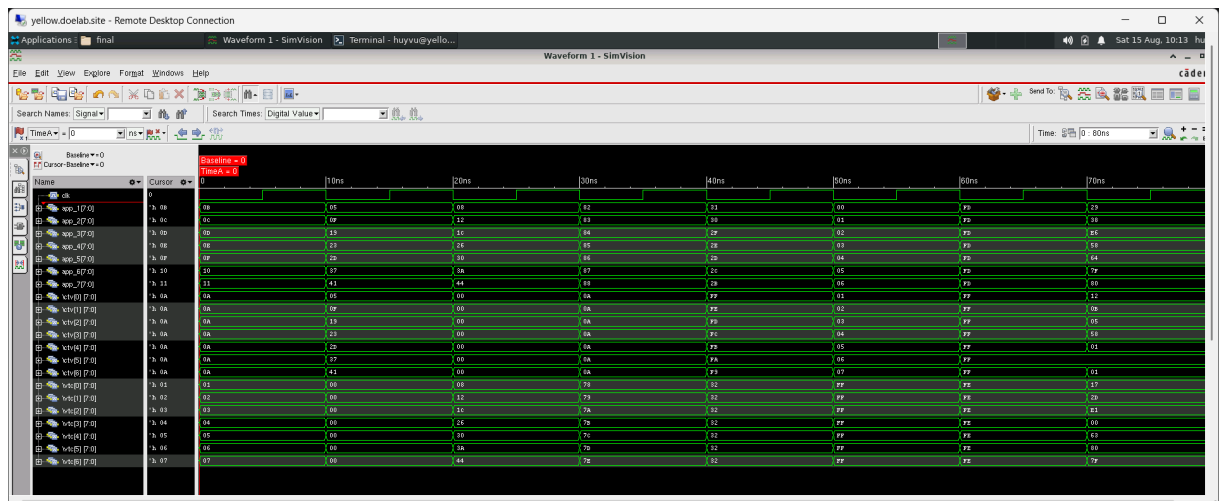
- Hiệu năng LVT: Các cell LVT cải thiện đáng kể tốc độ chuyển mạch. Trong cùng điều kiện PVT, LVT giúp giảm Data Path delay từ 28% đến 62% so với HVT.
- Khả năng đạt timing: Chỉ riêng dòng cell LVT ở góc Fast Corner mới đáp ứng được yêu cầu timing (Slack dương +1 ps và +6 ps). Toàn bộ các cấu hình sử dụng cell HVT đều vi phạm timing.

Tác động của điều kiện PVT (Điện áp & Nhiệt độ)

- Góc Slow Corner (0.9V, 125°C): Đây là điều kiện khắc nghiệt nhất. Điện áp sụt giảm xuống 0.9V kết hợp nhiệt độ cao làm suy giảm mạnh dòng lái (I_{on}) của MOSFET, dẫn đến Data Path kéo dài cực đại (8374 ps ở HVT và 3166 ps ở LVT), gây vi phạm slack nghiêm trọng (lên tới -7174 ps).
- Góc Fast Corner (1.1V/1.32V, 0°C): Điện áp cao làm tăng tốc độ đáp ứng của các cổng logic, đưa Data Path về mức dao động 1194 ps – 1708 ps.

4.7 Khối CTV APP Calculation

Bước đầu ta sẽ khảo sát dạng sóng waveform của thiết kế khối này.



Hình 4.7: Dạng sóng của khối CTV APP Calculation

- Giai đoạn cộng số nguyên cơ bản (0 ns - 10 ns): Tín hiệu ctv duy trì hằng số 8'h0A (10 thập phân) trên cả 7 kênh, còn vtc tăng dần từ 8'h01 đến 8'h07. Bus ngõ ra app

phản ánh tức thì giá trị tổng tương ứng từ 8'h0B đến 8'h11. Mạch đáp ứng chuẩn xác tính chất cộng tuyến tính $app_i = vtc_i + 10$.

- Giai đoạn kiểm tra phần tử trung tính (10 ns - 30 ns): Từ 10 ns - 20 ns, vtc bị kéo xuống 8'h00. Ngõ ra app sao chép hoàn toàn giá trị của ctv. Từ 20 ns - 30 ns, ctv bị kéo xuống 8'h00. Ngõ ra app sao chép hoàn toàn giá trị của vtc. Mạch xác nhận không có hiện tượng sai lệch dữ liệu hay gán nhầm mức logic khi một trong hai vector ngõ vào bằng 0.
- Giai đoạn xử lý số âm bù 2 và hiện tượng tràn bit (40 ns - 60 ns): Tại 40 ns, ctv mang các giá trị từ 8'hFF giảm dần đến 8'hF9. Phép cộng với vtc = 8'h32 cho ra kết quả app giảm dần từ 8'h31 đến 8'h2B, thể hiện đúng bản chất phép trừ trong hệ thống Bù 2. Tại 50 ns, vtc đạt giá trị cực đại 8-bit 8'hFF. Khi cộng thêm 8'h01, kết quả $255 + 1 = 256$ bị tràn bit nhớ *Co*, phần dữ liệu 8-bit còn lại của app_1 xoay vòng về 8'h00. Tương tự, app_7 nhận $255 + 7 = 262 \Rightarrow 8'h06$.

Library Name	Total Area	Total Power
fast_vdd1v0_basicCells_hvt	404.586	3.68958e-04
fast_vdd1v2_basicCells_hvt	404.586	5.34551e-04
slow_vdd1v0_basicCells_hvt	821.826	3.34493e-04
slow_vdd1v2_basicCells_hvt	569.772	4.02237e-04
fast_vdd1v0_basicCells_lvt	404.586	3.80819e-04
fast_vdd1v2_basicCells_lvt	275.310	5.38660e-04
slow_vdd1v0_basicCells_lvt	483.588	2.77465e-04
slow_vdd1v2_basicCells_lvt	483.588	3.97555e-04

Bảng 4.13: Tổng hợp thông số Diện tích và Công suất của khối Min/Submin Calculation qua các process corners.

Phân tích Diện tích (Total Area):

- Tác động của góc PVT và sụt áp: Diện tích mạch đạt giá trị lớn nhất ở góc slow_vdd1v0_basicCells_hvt (821.826). Ở điều kiện sụt áp (1.0V/0.9V) và nhiệt độ cao, độ trễ linh kiện tăng mạnh, buộc công cụ tổng hợp phải tăng kích thước cell và chèn thêm nhiều buffer/cổng logic phụ để thỏa mãn ràng buộc thời gian.
- Hiệu quả tối ưu diện tích của cell LVT: Ở góc Slow 1.0V, thư viện LVT giúp giảm diện tích xuống 483.588 (giảm 41.2% so với HVT). Do dòng lái của LVT mạnh hơn ở điện áp thấp, công cụ tổng hợp không cần gia tăng kích thước cell quá lớn để đạt timing.
- Cực tiểu diện tích: Diện tích nhỏ nhất đạt được ở góc fast_vdd1v2_basicCells_lvt (275.310), giảm 31.9% so với mức tiêu chuẩn 404.586 (ở các góc Fast 1.0V). Điện áp cao (1.2V) kết hợp với V_{th} thấp giúp các cổng logic kích thước tối thiểu vẫn đảm bảo tốc độ chuyển mạch.

Phân tích Công suất tiêu thụ (Total Power):

- Sự phụ thuộc vào Điện áp cấp (V_{dd}): Điện áp là yếu tố chi phối chính tới tổng công suất tiêu thụ do thành phần công suất động tỉ lệ thuận với V_{dd}^2 . Khi tăng điện áp từ 1.0V lên 1.2V: Trong thư viện HVT (góc Fast): Công suất tăng từ 3.68958×10^{-4} lên 5.34551×10^{-4} (tăng 44.9%). Trong thư viện LVT (góc Fast): Công suất tăng

từ 3.80819×10^{-4} lên 5.38660×10^{-4} (tăng 41.4%).

- So sánh công suất HVT và LVT: Tại góc Fast (1.0V): Công suất của LVT (3.80819×10^{-4}) cao hơn HVT (3.68958×10^{-4}) do điện áp ngưỡng thấp làm tăng dòng rò (leakage power). Tại góc Slow (1.0V): Công suất của LVT (2.77465×10^{-4}) lại thấp hơn HVT (3.34493×10^{-4}). Nguyên nhân do thiết kế HVT ở góc này có diện tích tăng vọt (821.826 so với 483.588), làm tăng điện dung chuyển mạch tổng và kéo theo công suất động tăng cao.
- Công suất thấp nhất: Đạt 2.77465×10^{-4} ở góc slow_vdd1v0_basicCells_lvt nhờ giảm thiểu điện áp và quy mô diện tích gọn nhẹ. Công suất cao nhất: Đạt 5.38660×10^{-4} ở góc fast_vdd1v2_basicCells_lvt do hoạt động ở mức điện áp đỉnh (1.2V/1.32V) kết hợp tần số đồng cắt cao.

STT	Góc PVT	Đường đi tín hiệu (Critical Path)	Độ trễ (ps)	Thời gian dư (ps)
1	fast_1v0_hvt	ctv_7[0] → ADDHX2HVT → ADDFX2HVT (×6) → XNOR2X2HVT → app_7[7]	687	463
2	fast_1v2_hvt	ctv_7[0] → ADDHX2HVT → ADDFX2HVT (×6) → XNOR2X2HVT → app_7[7]	488	662
3	slow_1v0_hvt	ctv_7[0] → CLKAND2X6HVT → INVX4HVT → OAI2I1X4HVT → NAND2X2HVT (×2) → OAI2BB1X1HVT → NAND2X2HVT (×2) → OAI2BB1X1HVT → NAND2X1HVT (×2) → XNOR2X1HVT → BUFX16HVT → app_7[7]	1692	-542
4	slow_1v2_hvt	ctv_7[0] → INVX1HVT → NOR2BX1HVT → (OAI2I1X1HVT → OAI2BB1X1HVT) (×6) → CLKMX2X6HVT → app_7[7]	1029	121
5	fast_1v0_lvt	ctv_7[0] → ADDHX2LVT → ADDFX2LVT (×6) → XNOR2X2LVT → app_7[7]	383	767
6	fast_1v2_lvt	ctv_7[0] → ADDHX1LVT → ADDFX1LVT (×6) → XNOR2X1LVT → app_7[7]	336	814
7	slow_1v0_lvt	ctv_7[0] → AND2XLLVT → ADDFX4LVT → OAI2I1X1LVT → OAI2BB1X1LVT → ADDFX4LVT (×3) → ADDHX4LVT → XNOR2X4LVT → app_7[7]	1122	28
8	slow_1v2_lvt	ctv_7[0] → ADDHX1LVT → ADDFX4LVT (×6) → XNOR2X4LVT → app_7[7]	733	417

Bảng 4.14: Thống kê độ trễ các góc phân tích (PVT corners) cho khối ctv_app_calc.

- Tác động của Điện áp cấp (V_{dd}): Việc nâng điện áp hoạt động từ 1.0V lên 1.2V làm giảm đáng kể độ trễ đường truyền dữ liệu và tăng lượng thời gian dư trên mọi góc PVT. Ví dụ ở điều kiện Fast HVT, thời gian trễ giảm từ 687 ps xuống 488 ps (Slack tăng từ 463 ps lên 662 ps).
- So sánh hiệu năng HVT và LVT: Thư viện LVT (V_{th} thấp) cho tốc độ đáp ứng chuyển mạch vượt trội so với HVT. Tại góc fast_1v0, LVT giảm trễ tới 44.2% so với HVT (383 ps so với 687 ps). Đặc biệt ở điều kiện Slow 1.0V, LVT vẫn thỏa mãn thời gian đáp ứng (Slack = 28 ps), trong khi HVT không đáp ứng được yêu cầu (Slack = -542 ps).
- Góc tối hạn: Góc slow_1v0_hvt là trường hợp xấu nhất gây vi phạm thời gian nghiêm trọng (Slack âm -542 ps, Data Path kéo dài đến 1692 ps). Ở điều kiện nhiệt độ cao (125°C) và điện áp thấp, công cụ tổng hợp bị chèn chuỗi phức tạp các cổng logic nhỏ (CLKAND2, NAND2, OAI2BB1, BUFX16) làm tăng số tầng logic (logic depth).

5. Kết luận và hướng phát triển

5.1 Kết luận

Trong quá trình thực hiện đề tài, em đã nghiên cứu các kiến thức cơ bản về mã LDPC, mã QC-LDPC và nguyên lý của thuật toán giải mã Min-Sum, từ đó phân tích các yêu cầu cần thiết để hiện thực thuật toán trên phần cứng FPGA. Trên cơ sở đó, kiến trúc bộ giải mã được phân tích và phân chia thành các khối chức năng phù hợp với đặc điểm xử lý song song và yêu cầu lưu trữ dữ liệu của thuật toán.

Về mặt thiết kế, em đã hoàn thành việc xây dựng các khối phần cứng cần thiết cho kiến trúc bộ giải mã bằng SystemVerilog. Việc phân chia thiết kế thành các khối chức năng độc lập cũng giúp thuận lợi cho quá trình kiểm tra, mô phỏng và tổng hợp. Kết quả mô phỏng RTL cho thấy các chức năng chính của thiết kế hoạt động đúng theo nguyên lý thuật toán đã phân tích. Các quá trình lưu trữ, truyền và xử lý dữ liệu trong kiến trúc được kiểm tra thông qua các trường hợp mô phỏng khác nhau, qua đó bước đầu xác nhận tính đúng đắn về mặt chức năng của thiết kế. Bên cạnh kiểm tra chức năng, em cũng đã thực hiện quá trình tổng hợp và đánh giá thiết kế tại nhiều điều kiện PVT khác nhau. Các thông số như diện tích phần cứng, độ trễ đường truyền và timing slack đã được khảo sát nhằm đánh giá khả năng đáp ứng của kiến trúc trong những điều kiện hoạt động khác nhau. Kết quả cho thấy điện áp nguồn, nhiệt độ và loại thư viện cell có ảnh hưởng đáng kể đến đặc tính timing của thiết kế. Đặc biệt, các điều kiện điện áp thấp và nhiệt độ cao tạo ra những trường hợp bất lợi hơn về mặt thời gian, trong khi việc sử dụng thư viện LVT có thể cải thiện đáng kể tốc độ hoạt động.

Tuy nhiên, do thời gian thực hiện đề tài còn hạn chế và tiến độ thực hiện tương đối gấp rút, kết quả hiện tại vẫn còn một số điểm chưa hoàn thiện. Thiết kế mới chủ yếu được kiểm chứng thông qua mô phỏng RTL và quá trình tổng hợp, chưa được triển khai hoàn chỉnh lên FPGA kit thực tế. Vì vậy, các kết quả về khả năng hoạt động trong môi trường phần cứng thực, tốc độ xử lý thực tế, mức sử dụng tài nguyên FPGA và công suất tiêu thụ thực tế vẫn chưa được kiểm chứng đầy đủ. Bên cạnh đó, một số kết quả phân tích timing tại các góc PVT bất lợi vẫn còn tồn tại vi phạm timing. Điều này cho thấy thiết kế vẫn cần được tiếp tục tối ưu về độ sâu logic, cấu trúc xử lý song song và đường truyền dữ liệu trước khi có thể đạt được hiệu năng mục tiêu trong mọi điều kiện hoạt động. Ví dụ, kết quả phân tích cho thấy tại điều kiện slow, điện áp thấp và nhiệt độ cao, một số đường dữ liệu có thể xuất hiện slack âm và trở thành các đường tới hạn của thiết kế.

Nhìn chung, mặc dù chưa thể hoàn thiện toàn bộ các bước của một quy trình triển khai phần cứng hoàn chỉnh, đề tài đã đạt được các mục tiêu chính trong phạm vi thời gian cho phép, đồng thời giúp em có thêm kinh nghiệm thực tế trong việc nghiên cứu thuật toán, thiết kế phần cứng số, mô phỏng RTL và sử dụng các công cụ EDA phục vụ cho quá trình tổng hợp và đánh giá thiết kế.

5.2 Định hướng phát triển

Trong thời gian tới, hướng phát triển đầu tiên của đề tài là tiếp tục hoàn thiện việc tích hợp các thành phần đã thiết kế thành một hệ thống giải mã QC-LDPC hoàn chỉnh ở mức top-level. Khi đó, toàn bộ luồng dữ liệu từ đầu vào, quá trình giải mã lặp cho đến dữ liệu đầu ra sẽ được kiểm tra thống nhất thay vì chỉ đánh giá từng thành phần ở mức riêng lẻ.

Một hướng phát triển quan trọng là triển khai thiết kế lên FPGA kit thực tế. Đây là bước cần thiết để kiểm chứng khả năng hoạt động của kiến trúc trong môi trường phần cứng thực tế. Sau khi triển khai, có thể tiến hành đo đặc tần số hoạt động, throughput, latency, mức sử dụng tài nguyên và công suất tiêu thụ, từ đó đánh giá chính xác hơn hiệu quả của kiến trúc so với các kết quả thu được từ mô phỏng và tổng hợp.

Bên cạnh đó, thiết kế cần tiếp tục được tối ưu về timing nhằm loại bỏ các đường truyền còn vi phạm tại những góc PVT bất lợi. Có thể nghiên cứu các phương pháp như giảm độ sâu logic, tối ưu cấu trúc tính toán, cân bằng đường truyền hoặc áp dụng pipeline ở những vị trí phù hợp. Mục tiêu là nâng cao tần số hoạt động nhưng vẫn duy trì diện tích và công suất ở mức hợp lý.

Về mặt kiến trúc, có thể tiếp tục nghiên cứu phương pháp tổ chức bộ nhớ và mức độ song song của quá trình xử lý nhằm tăng băng thông dữ liệu và throughput của bộ giải mã. Đồng thời, thiết kế có thể được mở rộng theo hướng tham số hóa để hỗ trợ nhiều cấu hình QC-LDPC khác nhau, giúp tăng tính linh hoạt và khả năng ứng dụng của kiến trúc.

Như vậy, kết quả đạt được trong đề tài có thể xem là bước nền tảng cho quá trình phát triển một kiến trúc giải mã QC-LDPC hoàn chỉnh trên FPGA. Mặc dù vẫn còn những hạn chế do thời gian thực hiện và chưa triển khai trên kit thực tế, các kết quả về thiết kế, mô phỏng và đánh giá phần cứng đã tạo tiền đề để tiếp tục tối ưu và hoàn thiện hệ thống trong các giai đoạn nghiên cứu tiếp theo.

Tài liệu tham khảo

- [1] S.-Y. Chung, G. Forney, T. Richardson **and** R. Urbanke, “On the design of low-density parity-check codes within 0.0045 dB of the Shannon limit,” **journal** 5, **number** 2, **pages** 58–60, 2001. DOI: 10.1109/4234.905935
- [2] C. Kun, S. Qi, L. Shengkai **and** P. Chengzhi, “Implementation of encoder and decoder for LDPC codes based on FPGA,” **journal** 30, **number** 4, **pages** 642–650, 2019. DOI: 10.21629/JSEE.2019.04.02
- [3] S. Mhaske, D. Uliana, H. Kee, T. Ly, A. Aziz **and** P. Spasojevic, “A 2.48Gb/s FPGA-based QC-LDPC decoder: An algorithmic compiler implementation,” 2015, **pages** 88–93. DOI: 10.1109/SARNOF.2015.7324649
- [4] S. Nimara, O. Boncalo, A. Amaricai **and** M. Popa, “FPGA architecture of multi-codeword LDPC decoder with efficient BRAM utilization,” 2016, **pages** 1–4. DOI: 10.1109/DDECS.2016.7482452
- [5] R. G. Gallager, “Low-Density Parity-Check Codes,” Cambridge, MA: M.I.T. Press, 1963.
- [6] D. MacKay **and** R. Neal, “Near Shannon limit performance of low density parity check codes,” *Electronics Letters*, **journal** 32, **pages** 1645–1646, 18 1996. DOI: 10.1049/el:19961141 eprint: <https://digital-library.theiet.org/doi/pdf/10.1049/el%3A19961141>. url: <https://digital-library.theiet.org/doi/abs/10.1049/el%3A19961141>
- [7] K. Zhang, X. Huang **and** Z. Wang, “High-throughput layered decoder implementation for quasi-cyclic LDPC codes,” *IEEE J.Sel. A. Commun.*, **journal** 27, **number** 6, **pages** 98–994, **august** 2009, ISSN: 0733-8716. DOI: 10.1109/JSAC.2009.090816 url: <https://doi.org/10.1109/JSAC.2009.090816>
- [8] S. J. Johnson, “Introducing Low-Density Parity-Check Codes,” 2008, **page** 49. url: <https://api.semanticscholar.org/CorpusID:17943867>

Mã nguồn toàn bộ chương trình: Link